

Tema

MANUAL DE CONFIGURACIÓN SISTEMA CONVERSIÓN UNIDADES MONSTER CON SOAP SIN BDD EN JAVA

Autores

ALEJANDRO CAETANO FLORES MAYA

ANTHONY ALAIN MORALES CARVAJAL

LEONARDO VINICIO NARVÁEZ CRIOLLO

Tutor

Ing. Eduardo Mauricio Campaña Ortega

MIS.MDU.CCNA. CCIA.

PhD. (c) Ingeniería de Software

PhD. (c) Seguridad Información

Fecha

28/10/2026

MANUAL DE CONFIGURACIÓN SISTEMA CONVERSIÓN DE UNIDADES CON SOAP SIN BDD EN JAVA

1 INTRODUCCIÓN **5**

2 OBJETIVOS **5**

2.1 Objetivo General 5

2.2 Objetivos Específicos 5

3 MARCO TEÓRICO **6**

3.1 Web Services SOAP y WSDL 6

3.2 Arquitectura SOA 6

3.3 Patrón de Arquitectura Modelo-Vista-Controlador (MVC) 6

3.4 Servidor Payara y JAX-WS 7

4 DESARROLLO DEL APLICATIVO **7**

4.1 Servidor Web SOAP 7

4.1.1 Paso 1: Creación de la Instancia de Servidor Payara en el IDE 8

4.1.2 Paso 2: Selección de Versión y Directorio de Instalación 8

4.1.3 Paso 3: Configuración de Dominio y Puertos de Red 9

4.1.4 Paso 4: Creación del Proyecto Tipo Aplicación Web 9

4.1.5 Paso 5: Parametrización de Metadatos del Proyecto 10

4.1.6 Paso 6: Asociación del Servidor y Plataforma Jakarta EE 10

4.1.7 Paso 7: Estructuración de Paquetes del Servidor 11

4.1.8 Paso 8: Creación del Web Service JAX-WS 11

4.1.9 Paso 9: Definición del Endpoint Central 12

4.1.10 Paso 10: Implementación de la Lógica del Servicio Web 12

4.1.11 Paso 11: Implementación del Interceptor de Seguridad (SOAPHandler) 15

Para desacoplar la validación de seguridad de la lógica matemática sin utilizar una base de datos, se implementa la clase ManejadorAutenticacion que extiende de SOAPHandler<SOAPMessageContext>: 15

4.1.12 Paso 12: Implementación de la Capa de Dominio y Conversión 17

4.1.13 Paso 13: Despliegue en Servidor y Comprobación del WSDL 25

4.2 Cliente Consola 25

4.2.1 Paso 1: Creación del Proyecto Maven en NetBeans 25

4.2.2 Paso 2: Consumo del WSDL mediante el Asistente Gráfico Web Service Client 26

4.2.3 Paso 3: Estructuración Modular de Paquetes (MVC) 27

4.2.4 Paso 4: Implementación de la Capa de Modelo (ConversorService) 28

4.2.5 Paso 5: Implementación de Controladores y Vista de Consola 29

4.3 Cliente Escritorio 36

4.3.1 Paso 1: Creación del Proyecto Gráfico en NetBeans 36

4.3.2 Paso 2: Generación de Clientes Proxy mediante el Asistente Web Service Client 37

4.3.3 Paso 3: Estructuración de Paquetes MVC 38

4.3.4 Paso 4: Implementación de la Capa de Modelo (ConversorService) 38

4.3.5 Paso 5: Diseño de Vistas con Formularios Swing 39

4.3.6 Paso 6: Implementación del Controlador de Eventos (AppControlador) 40
4.4 Cliente Móvil 42
4.4.2 Paso 2: Estructuración Modular bajo el Patrón MVC 43
4.4.3 Paso 3: Implementación de la Capa de Modelo y Despacho SOAP Asíncrono 44
4.4.4 Paso 4: Diseño de Layouts XML 47
4.4.5 Paso 5: Implementación de Actividades y Ejecución Móvil 57
4.5.1 Paso 1: Creación del Proyecto Web Jakarta Faces en NetBeans 59
4.5.2 Paso 2: Configuración del Entorno JSF y Descriptores 59
4.5.3 Paso 3: Consumo del WSDL mediante el Asistente Web Service Client 61
4.5.4 Paso 4: Implementación de la Capa de Modelo (ConversorService) 61
4.5.5 Paso 5: Implementación de Managed Beans Controladores con CDI 62
4.5.6 Paso 6: Diseño de Vistas Web con Facelets (XHTML) 66
5 CONCLUSIONES 71
6 RECOMENDACIONES 72
7 REFERENCIAS BIBLIOGRÁFICAS 72

ÍNDICE DE ILUSTRACIONES

Figura 1
Creación de Servidor Payara 9
Figura 2
Configuración básica del servidor Payara 9
Figura 3
Configuración avanzada del servidor Payara 10
Figura 4
Creación de Proyecto Tipo Aplicación Web. 10
Figura 5
Parámetros del proyecto servidor. 11
Figura 6
Selección de Servidor y perfil Jakarta EE. 11
Figura 7
Estructura modular de paquetes del servidor. 12
Figura 8
Creación de Web Service JAX-WS. 12
Figura 9
Configuración de anotaciones del servicio web. 13
Figura 10
Creación de clase Message Handler 16
Figura 11
Contrato WSDL publicado en Payara Server. 26
Figura 13
Asistente Web Service Client para consumo de WSDL en NetBeans. 28
Figura 14
Configuración de jaxws-maven-plugin y referencias WSDL generadas 28
Figura 15

Estructura de paquetes MVC del Cliente Consola en NetBeans 29
Figura 16
Ejecución del Cliente Consola (Inicio de sesión en terminal) 36
Figura 17
Ejecución de operaciones de conversión en Cliente Consola 37
Figura 18
Creación del Proyecto Java Swing en NetBeans 37
Figura 19
Asistente Web Service Client en Cliente Escritorio 38
Figura 20
Artefactos generados en xml-resources y paquetes de stubs SOAP 39
Figura 21
Diseño visual del formulario de login en Swing (LoginView) 41
Figura 22
Diseño visual de la ventana principal del conversor (ConversorView) 41
Figura 23
Interfaz gráfica de Login ejecutada en Cliente Escritorio 43
Figura 24
Operación de conversión exitosa en Cliente Escritorio 43
Figura 25
Configuración de permisos de red en AndroidManifest.xml 44
Figura 26
Estructura de paquetes MVC 45
Figura 27
Diseño visual de layout de login 49
Figura 28
Diseño visual de layout del conversor 50
Figura 29
Ejecución del Cliente Móvil (Login) 59
Figura 30
Ejecución del Cliente Móvil (Conversor) 59
Figura 31
Creación del proyecto Web Jakarta Face en Netbeans 60
Figura 32
Descriptores de despliegue web.xml, beans.xml y faces-config.xml 61
Figura 33
Asistente Web Service Client en Proyecto Web 62
Figura 34
Interfaz Web de Login en navegador 72
Figura 35
Interfaz Web del Conversor con actualización 72

ÍNDICE DE TABLAS

Tabla 1
Librerías e importaciones del Web Service 13
Tabla 2
Anotación @WebService y declaración del Endpoint 14

Tabla 3
Inyección de componentes de servicio de dominio 14
Tabla 4
Métodos y operaciones expuestas en el servicio SOAP 14
Tabla 5
Configuración de descriptor manejadores.xml. 17
Tabla 6
Interceptor ManejadorAutenticacion.java. 17
Tabla 7
Interfaz Conversor (IConversor.java) 18
Tabla 8
Conversor Longitud (ConversorLongitud.java) 18
Tabla 9
Conversor Masa (ConversorMasa.java) 19
Tabla 10
Conversor Temperatura (ConversorTemperatura.java) 19
Tabla 11
ServicioConversor.java 20
Tabla 12
UnidadMapper.java 21
Tabla 13
AdministradorCredenciales.java 23
Tabla 14
AdministradorTokens.java 25
Tabla 15
ConversorService.java 29
Tabla 16
ConsolaView.java 30
Tabla 17
ConsolaControlador.java 33
Tabla 18
AppControlador.java 35
Tabla 19
WSConversorConsoleClient.java 36
Tabla 20
ConversorService.java 39
Tabla 21
AppControlador.java 42
Tabla 22
Modelo de comuniación SOAP asíncrona móvil. 45
Tabla 23
Diseño XML de la vista de autenticación móvil. 50
Tabla 24
Diseño XML de la vista principal móvil 53
Tabla 25
Controlador de actividades móviles 58
Tabla 26
Descriptores de despliegue JSF 61
Tabla 27

Modelo de consumo SOAP en Cliente Web 63
Tabla 28
Managed Bean de vista reactiva 64
Tabla 29
Formulario Facelets de autenticación web 68
Tabla 30
Interfaz Facelets del conversor web reactivo 69

1 INTRODUCCIÓN

Un servicio web basado en SOAP (Simple Object Access Protocol) constituye un mecanismo estandarizado para la transferencia e interoperabilidad de información estructurada entre sistemas computacionales heterogéneos. A diferencia de estilos arquitectónicos basados en recursos, SOAP impone el uso del Lenguaje de Marcado Extensible (XML) como formato estricto de serialización de mensajes y se fundamenta en un contrato formal definido mediante WSDL (Web Services Description Language). Dicho contrato desacopla y especifica rigurosamente las operaciones disponibles, los tipos de datos y los parámetros de comunicación, proveyendo un marco con garantías formales de tipado, transaccionalidad y seguridad a nivel de mensaje, características requeridas en infraestructuras corporativas.

En el desarrollo de servicios web SOAP bajo la plataforma Java, la lógica de negocio se expone mediante puntos de enlace (endpoints) dentro de un modelo de Arquitectura Orientada a Servicios (SOA). Bajo este esquema, el servidor actúa como un proveedor centralizado de capacidades operativas que los clientes consumen a través del protocolo HTTP, una vez procesada la interfaz abstracta del servicio.

El despliegue de estos componentes se realiza sobre Payara Server, un servidor de aplicaciones de grado empresarial compatible con el estándar Jakarta EE y derivado de GlassFish. Payara provee soporte nativo para la especificación JAX-WS (Java API for XML Web Services), lo que permite gestionar el ciclo de vida de los servicios web, automatizar la generación de contratos WSDL, supervisar el rendimiento de la aplicación y garantizar la escalabilidad dentro de entornos distribuidos.

2 OBJETIVOS

2.1 Objetivo General

Desarrollar un manual de documentación que explique la construcción de un sistema conversor de unidades de medida (longitud, masa y temperatura) basado en una arquitectura cliente-servidor y el patrón arquitectónico MVC, integrando cuatro clientes nativos (escritorio en Swing, consola en CLI, web en JSF y móvil en Android) con un servidor SOAP implementado en Java, sin base de datos (con autenticación por tokens en memoria gestionada mediante cabeceras SOAP) y alojado en Payara Server.

2.2 Objetivos Específicos

- Describir detalladamente el diseño e implementación del servicio SOAP utilizando la especificación JAX-WS sobre Payara Server, incluyendo la generación automática del contrato WSDL y la estructuración modular de los conversores de longitud, masa y temperatura.
- Implementar un mecanismo de seguridad y autenticación sin base de datos mediante interceptores SOAP (SOAPHandler) para la validación y gestión de tokens de sesión transmitidos en los encabezados XML (SOAP Header).
- Proporcionar guías paso a paso para el desarrollo y consumo del servicio en las cuatro plataformas cliente (escritorio, consola, web y móvil), detallando el desacoplamiento entre vista, controlador y modelos generados a partir del WSDL.

- Explicar los fundamentos de la arquitectura SOA y las ventajas de utilizar Payara Server en términos de interoperabilidad, validación estricta de esquemas y procesamiento de mensajes en entornos heterogéneos.

3 MARCO TEÓRICO

3.1 Web Services SOAP y WSDL

Un Web Service SOAP (Simple Object Access Protocol) es un protocolo basado en XML diseñado para el intercambio de información estructurada en entornos de red descentralizados y distribuidos. La estructura de un mensaje SOAP está compuesta por un elemento raíz denominado Envelope, un Header (encabezado utilizado para el transporte de metadatos de control, autenticación y contexto de sesión) y un Body (cuerpo principal que encapsula la carga útil con las solicitudes, respuestas o fallos del servicio).

La interoperabilidad independiente de la plataforma se garantiza mediante el WSDL (Web Services Description Language). El WSDL es un contrato formal en XML que describe las operaciones del servicio, la tipología de datos aceptada (XSD) y los puntos de enlace físicos (endpoints). A partir de este contrato, los clientes generan de manera determinista los artefactos proxy (stubs) necesarios para invocar las operaciones remotas.

3.2 Arquitectura SOA

La Arquitectura Orientada a Servicios (SOA) es un modelo de diseño de software sustentado en la interacción de servicios desacoplados, autónomos y reutilizables. Cada servicio representa una función de negocio delimitada (como la conversión de una magnitud física o la autenticación de credenciales) y expone una interfaz estandarizada que abstrae la tecnología y el estado de su implementación interna.

En SOA, la comunicación entre componentes independientes se realiza mediante protocolos estándar; SOAP actúa como el protocolo de intercambio de mensajes tipados y WSDL como el contrato formal de especificación, asegurando un bajo acoplamiento y facilitando la evolución del sistema.

3.3 Patrón de Arquitectura Modelo-Vista-Controlador (MVC)

El patrón arquitectónico MVC segrega los componentes funcionales de una aplicación en tres capas desacopladas:

- **Modelo:** Representa las estructuras de datos, las reglas de validación y la lógica de negocio local. En los clientes consumidores de servicios SOAP, el modelo incluye los clientes proxy generados a partir del WSDL, los adaptadores de servicio y las entidades de datos locales.
- **Vista:** Corresponde a la interfaz de usuario encargada de proyectar la información procesada y capturar las entradas del usuario según la plataforma (vistas JFrame en

escritorio, interfaces terminal en consola, páginas Facelets/XHTML en web y vistas XML en Android).

- **Controlador:** Actúa como intermediario entre la vista y el modelo. Captura los eventos generados por el usuario, valida las entradas, orquesta la invocación de las operaciones del cliente SOAP y actualiza la vista con el resultado de la conversión o el estado de autenticación.

Este patrón garantiza la separación de responsabilidades, permitiendo que cuatro plataformas cliente con tecnologías visuales completamente distintas compartan la misma lógica de consumo del servicio web.

3.4 Servidor Payara y JAX-WS

Payara Server es una plataforma de software de grado empresarial diseñada para la ejecución robusta de aplicaciones Jakarta EE. Derivado de GlassFish, proporciona un entorno optimizado para servicios web tradicionales y microservicios, ofreciendo alta disponibilidad, administración centralizada y monitoreo de rendimiento.

Para el aprovisionamiento de servicios SOAP, Payara incorpora soporte nativo para JAX-WS (Java API for XML Web Services). Esta especificación permite definir servicios web mediante anotaciones (@WebService, @WebMethod), automatizando la serialización/deserialización de objetos Java a mensajes XML y publicando dinámicamente el contrato WSDL.

3.5 Interceptores SOAP (SOAP Handlers) y seguridad en Memoria

En arquitecturas SOAP, la especificación JAX-WS provee la interfaz SOAPHandler, la cual actúa como un filtro o interceptor que procesa los mensajes entrantes y salientes antes de que alcancen la lógica de negocio del servicio web. En sistemas sin base de datos (SIN BDD), este mecanismo permite desacoplar la seguridad: el interceptor inspecciona el SOAP Header para verificar la autenticidad de un token de seguridad generado en memoria y almacenado en estructuras concurrentes del servidor, denegando el acceso o permitiendo la ejecución de las funciones de conversión de forma transparente.

4 DESARROLLO DEL APLICATIVO

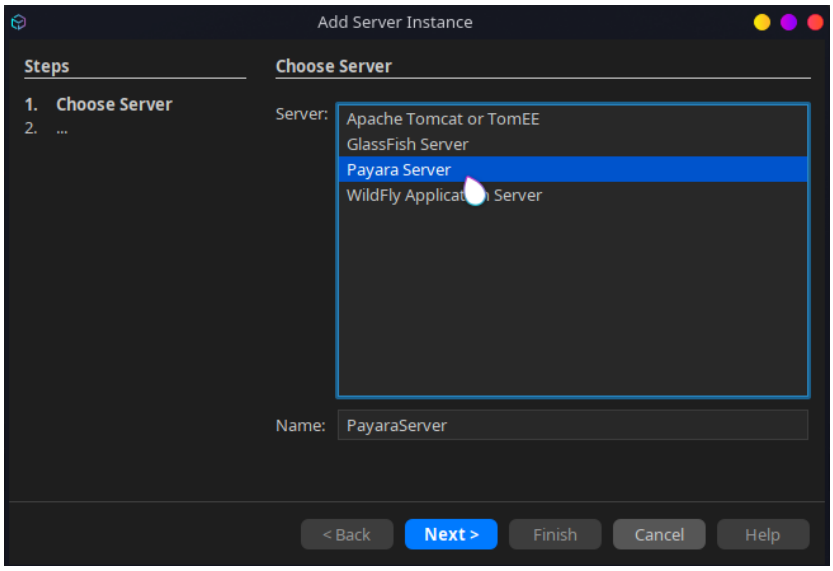
El desarrollo del sistema conversor de unidades distribuidas comprende la construcción del servidor de servicios web SOAP y la implementación de cuatro clientes nativos heterogéneos (consola, escritorio, móvil y web), desarrollados íntegramente sobre la plataforma Java y estructurados bajo el patrón arquitectónico Modelo-Vista-Controlador (MVC).

4.1 Servidor Web SOAP

4.1.1 Paso 1: Creación de la Instancia de Servidor Payara en el IDE

En el entorno de desarrollo integrado NetBeans, dentro del panel de administración de servicios (Services), se inicializa el asistente para el registro de un nuevo servidor de aplicaciones seleccionando la opción Payara Server.

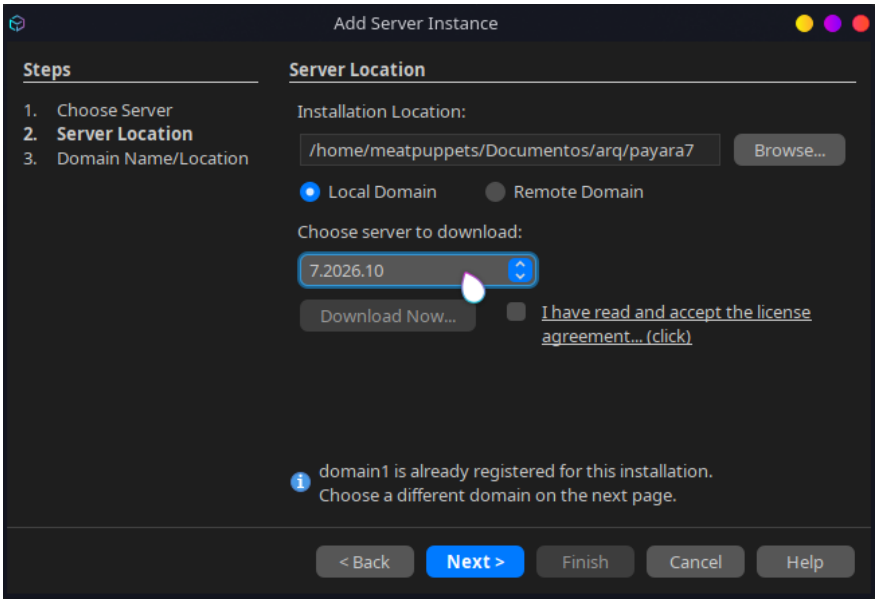
Figura 1
Creación de Servidor Payara



4.1.2 Paso 2: Selección de Versión y Directorio de Instalación

Se selecciona la versión estable de Payara Server Community compatible con la especificación Jakarta EE, se aceptan los términos de licenciamiento de código abierto y se define la ruta física del sistema de archivos donde se alojan los binarios del servidor.

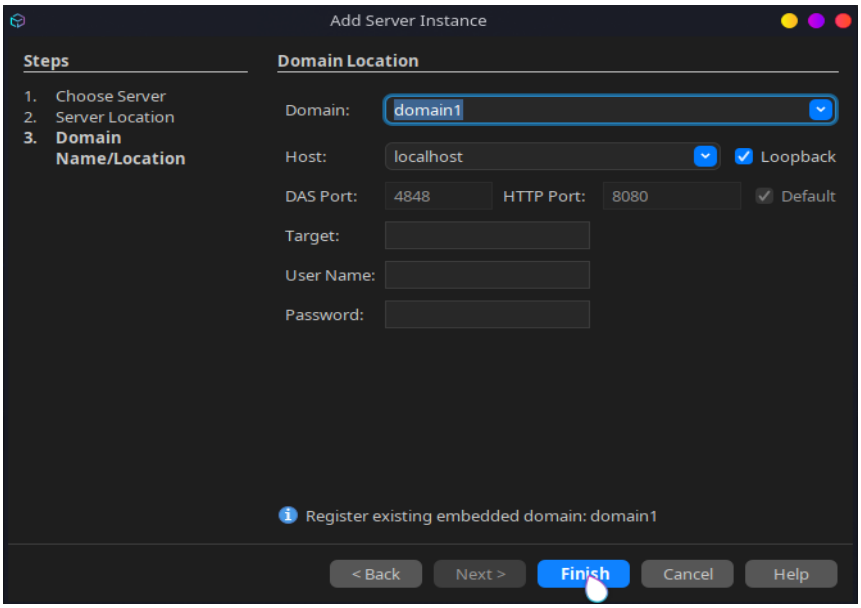
Figura 2
Configuración básica del servidor Payara



4.1.3 Paso 3: Configuración de Dominio y Puertos de Red

Se parametrizan las propiedades del dominio de ejecución (domain1), especificando el nombre de host de enlace local (localhost o interfaz de red configurada), el puerto del contenedor HTTP (8080) y el puerto de administración de la consola web (4848).

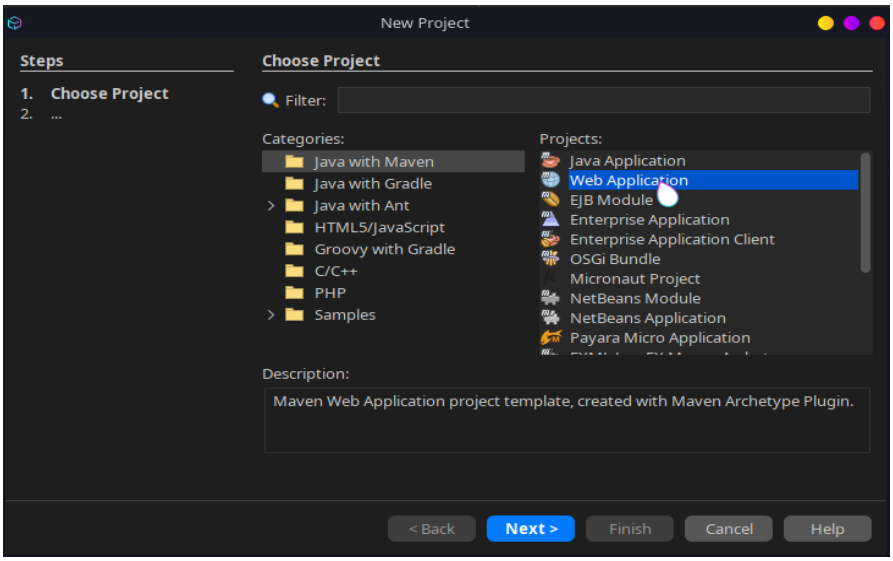
Figura 3
Configuración avanzada del servidor Payara



4.1.4 Paso 4: Creación del Proyecto Tipo Aplicación Web

Una vez activo el servidor de aplicaciones, se crea un nuevo proyecto en NetBeans clasificado como Java with Maven > Web Application, garantizando la gestión declarativa de dependencias y la compilación estandarizada mediante Maven.

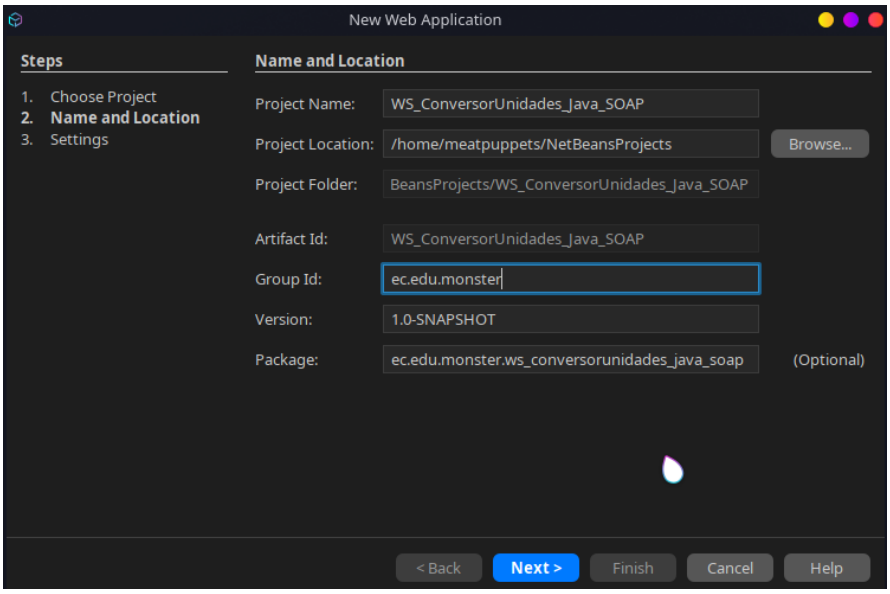
Figura 4
Creación de Proyecto Tipo Aplicación Web.



4.1.5 Paso 5: Parametrización de Metadatos del Proyecto

Se asigna el identificador de artefacto WS_ConversorUnidades_Java_SOAP, estableciendo el identificador de grupo (GroupId) bajo el estándar de nomenclatura inversa de dominio ec.edu.monster y la versión base 1.0-SNAPSHOT.

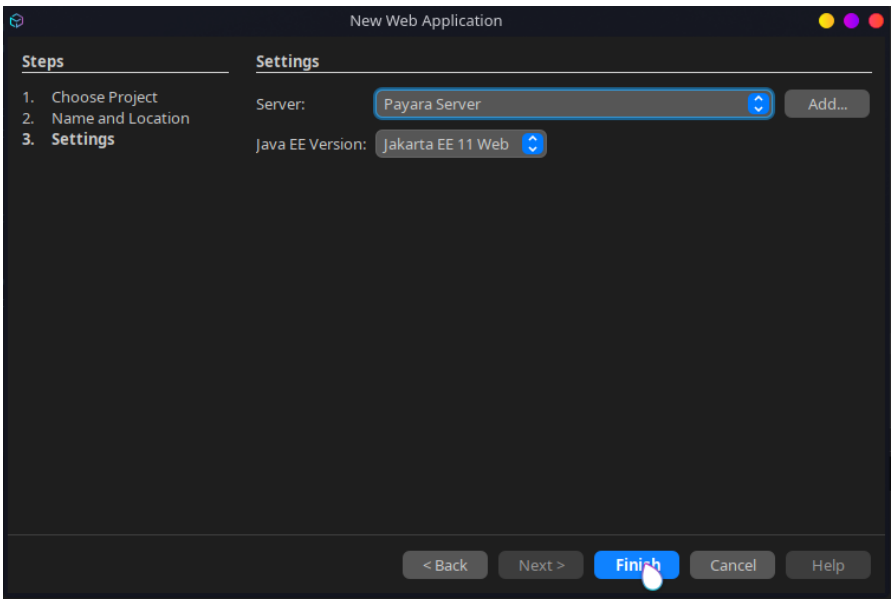
Figura 5
Parámetros del proyecto servidor.



4.1.6 Paso 6: Asociación del Servidor y Plataforma Jakarta EE

Se asocia el proyecto recién creado al servidor Payara configurado en los pasos previos, definiendo Jakarta EE 10 como el conjunto de especificaciones empresariales de desarrollo para asegurar el soporte nativo de JAX-WS y CDI.

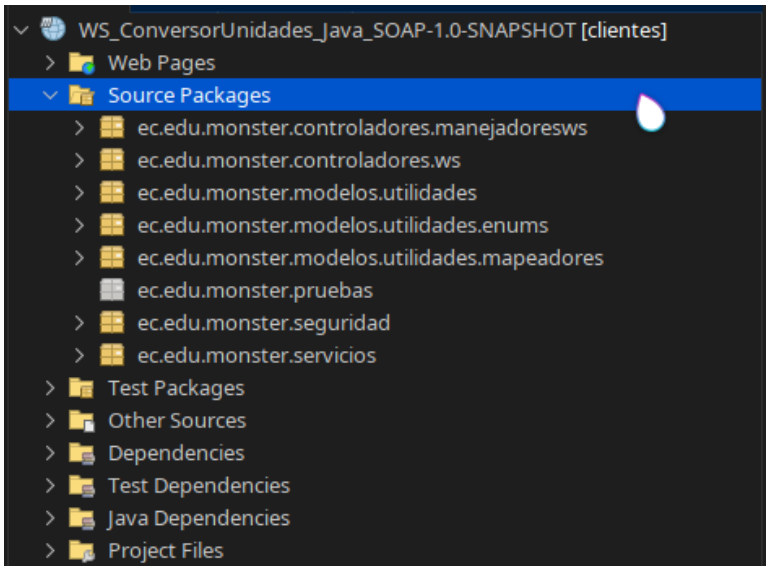
Figura 6
Selección de Servidor y perfil Jakarta EE.



4.1.7 Paso 7: Estructuración de Paquetes del Servidor

Para garantizar una separación de responsabilidades estricta, se organiza el código fuente dentro del directorio src/main/java en la siguiente estructura modular:

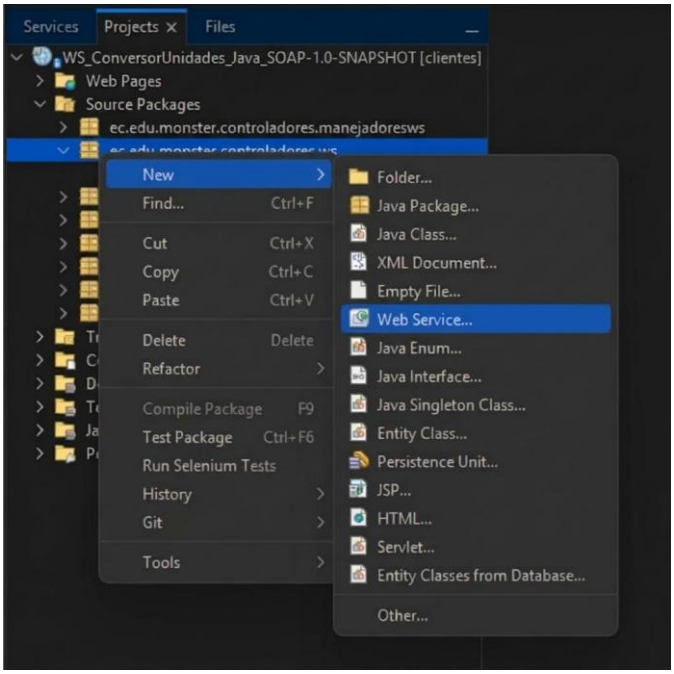
Figura 7
Estructura modular de paquetes del servidor.



4.1.8 Paso 8: Creación del Web Service JAX-WS

Dentro del paquete ec.edu.monster.controladores.ws, se genera un componente de tipo Web Service, el cual será administrado por el contenedor JAX-WS de Payara Server.

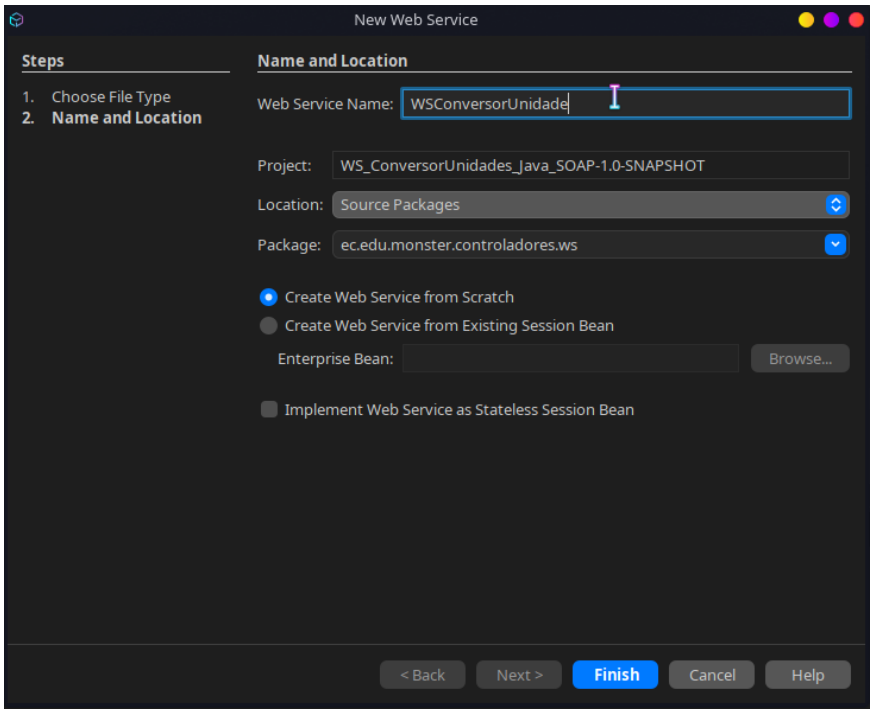
Figura 8
Creación de Web Service JAX-WS.



4.1.9 Paso 9: Definición del Endpoint Central

Se nombra la clase controladora del servicio como WSConversorUnidades, decorándola con anotaciones declarativas que establecen el nombre formal del servicio, el espacio de nombres de destino (targetNamespace) y el estilo de enlace.

Figura 9
Configuración de anotaciones del servicio web.



4.1.10 Paso 10: Implementación de la Lógica del Servicio Web

La clase WSConversorUnidades implementa la lógica de exposición remota a través de las siguientes definiciones formales:

1. Importaciones requeridas:

Tabla 1
Librerías e importaciones del Web Service

```
package ec.edu.monster.controladores.ws;

  import ec.edu.monster.modelos.utilidades.enums.UnidadLongitud;
  import ec.edu.monster.modelos.utilidades.enums.UnidadMasa;
  import ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura;
  import ec.edu.monster.modelos.utilidades.ConversorLongitud;
  import ec.edu.monster.modelos.utilidades.ConversorMasa;
  import ec.edu.monster.modelos.utilidades.ConversorTemperatura;
  import ec.edu.monster.modelos.utilidades.mapeadores.UnidadMapper;
  import ec.edu.monster.seguridad.AdministradorCredenciales;
  import ec.edu.monster.seguridad.AdministradorTokens;
  import ec.edu.monster.servicios.ServicioConversor;
  import jakarta.jws.WebService;
```

```
import jakarta.jws.WebMethod;
import jakarta.jws.WebParam;
import jakarta.jws.HandlerChain;
```

2. Definición del Endpoint y Atributos de Servicio:

Tabla 2
Anotación @WebService y declaración del Endpoint

```
@WebService(serviceName = "WSConversorUnidades")
  @HandlerChain(file = "manejadores.xml")
  public class WSConversorUnidades {
```

3. Métodos expuestos en el contrato WSDL:

- login: Valida usuario y contraseña en AdministradorCredenciales; ante éxito, retorna un token generado por AdministradorTokens.
- cambiarContrasenia: Actualiza la contraseña en memoria previa verificación de la clave actual.
- convertirLongitud, convertirMasa, convertirTemperatura: Mapean las cadenas recibidas a los enums respectivos mediante UnidadMapper y delegan el cálculo a la capa de servicios.

Tabla 3
Inyección de componentes de servicio de dominio

```
    private final ServicioConversor<UnidadLongitud> servicioLongitud =
      new ServicioConversor<>(new ConversorLongitud());
    private final ServicioConversor<UnidadMasa> servicioMasa =
      new ServicioConversor<>(new ConversorMasa());
    private final ServicioConversor<UnidadTemperatura> servicioTemperatura =
      new ServicioConversor<>(new ConversorTemperatura());
```

Tabla 4
Métodos y operaciones expuestas en el servicio SOAP

```
@WebMethod(operationName = "login")
  public String login(
    @WebParam(name = "usuario") String usuario,
    @WebParam(name = "contrasenia") String contrasenia) {
    if (AdministradorCredenciales.validarCredenciales(usuario, contrasenia)) {
      return AdministradorTokens.generarToken();
    }
    throw new RuntimeException("Credenciales incorrectas");
  }

  @WebMethod(operationName = "cambiarContrasenia")
  public String cambiarContrasenia(
    @WebParam(name = "contraseniaActual") String contraseniaActual,
```

```
        @WebParam(name = "contraseniaNueva") String contraseniaNueva) {
        AdministradorCredenciales.cambiarContrasenia(contraseniaActual, contraseniaNueva);
        return "Contraseña actualizada correctamente";
    }

    @WebMethod(operationName = "convertirLongitud")
    public double convertirLongitud(
        @WebParam(name = "valor") double valor,
        @WebParam(name = "unidadInicial") String unidadInicial,
        @WebParam(name = "unidadFinal") String unidadFinal) {
        UnidadLongitud origen = UnidadMapper.toLongitud(unidadInicial);
        UnidadLongitud destino = UnidadMapper.toLongitud(unidadFinal);
        return servicioLongitud.convertir(valor, origen, destino);
    }

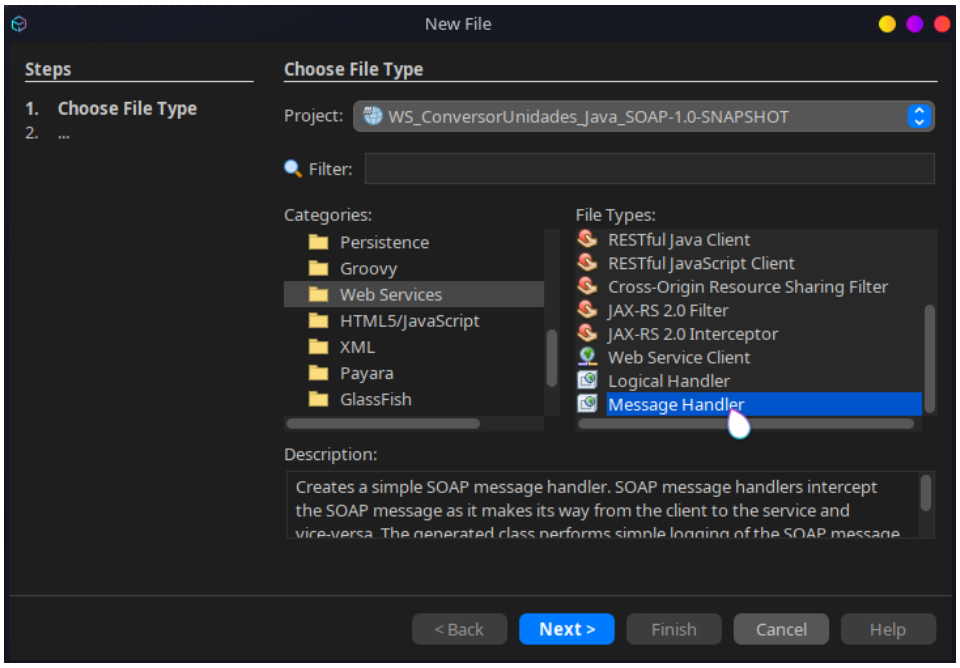
    @WebMethod(operationName = "convertirMasa")
    public double convertirMasa(
        @WebParam(name = "valor") double valor,
        @WebParam(name = "unidadInicial") String unidadInicial,
        @WebParam(name = "unidadFinal") String unidadFinal) {
        UnidadMasa origen = UnidadMapper.toMasa(unidadInicial);
        UnidadMasa destino = UnidadMapper.toMasa(unidadFinal);
        return servicioMasa.convertir(valor, origen, destino);
    }

    @WebMethod(operationName = "convertirTemperatura")
    public double convertirTemperatura(
        @WebParam(name = "valor") double valor,
        @WebParam(name = "unidadInicial") String unidadInicial,
        @WebParam(name = "unidadFinal") String unidadFinal) {
        UnidadTemperatura origen = UnidadMapper.toTemperatura(unidadInicial);
        UnidadTemperatura destino = UnidadMapper.toTemperatura(unidadFinal);
        return servicioTemperatura.convertir(valor, origen, destino);
    }
}
```

4.1.11 Paso 11: Implementación del Interceptor de Seguridad (SOAPHandler)

Para desacoplar la validación de seguridad de la lógica matemática sin utilizar una base de datos, se implementa la clase ManejadorAutenticacion que extiende de SOAPHandler<SOAPMessageContext>:

Figura 10
Creación de clase Message Handler



El manejador intercepta las solicitudes entrantes (!MESSAGE_OUTBOUND_PROPERTY), inspecciona el elemento <auth:token> del encabezado SOAP (SOAPHeader) y verifica su validez contra AdministradorTokens. Si el token no está activo o caducó, se aborta la petición lanzando una excepción SOAP.
Se crea el archivo descriptor manejadores.xml dentro del directorio de recursos para enlazar el interceptor a la cadena de ejecución del servicio:

Tabla 5
Configuración de descriptor manejadores.xml.

```
<?xml version="1.0" encoding="UTF-8"?>
  <handler-chains xmlns="https://jakarta.ee/xml/ns/jakartaee">
    <handler-chain>
      <handler>
        <handler-name>ManejadorAutenticacion</handler-name>          <handler-
class>ec.edu.monster.controladores.manejadoresws.ManejadorAutenticacion</handler-class>
      </handler>
    </handler-chain>
  </handler-chains>
```

Tabla 6
Interceptor ManejadorAutenticacion.java.

```
package ec.edu.monster.controladores.manejadoresws;

import ec.edu.monster.seguridad.AdministradorTokens;
import jakarta.xml.ws.handler.soap.SOAPHandler;
import jakarta.xml.ws.handler.soap.SOAPMessageContext;
import jakarta.xml.ws.handler.MessageContext;

import javax.xml.namespace.QName;
import org.w3c.dom.NodeList;
import java.util.Set;

public class ManejadorAutenticacion implements SOAPHandler<SOAPMessageContext> {

  @Override
  public boolean handleMessage(SOAPMessageContext contexto) {

    Boolean esSalida = (Boolean)
contexto.get(MessageContext.MESSAGE_OUTBOUND_PROPERTY);

    if (!esSalida) {
      try {

        QName operacion = (QName) contexto.get(MessageContext.WSDL_OPERATION);
        if (operacion != null && "login".equalsIgnoreCase(operacion.getLocalPart())) {
          return true;
        }

        var mensajeSOAP = contexto.getMessage();

        // Verificar si la llamada es al método login en el cuerpo SOAP
        if (mensajeSOAP != null && mensajeSOAP.getSOAPBody() != null) {
          var nodo = mensajeSOAP.getSOAPBody().getFirstChild();
          while (nodo != null && nodo.getNodeType() != org.w3c.dom.Node.ELEMENT_NODE) {
```

```
                nodo = nodo.getNextSibling();
            }
            if (nodo != null && "login".equalsIgnoreCase(nodo.getLocalName())) {
                return true;
            }
        }

        var encabezado = mensajeSOAP != null ? mensajeSOAP.getSOAPHeader() : null;

        if (encabezado == null) {
            throw new RuntimeException("Acceso denegado: Se requiere encabezado SOAP con el
token de seguridad.");
        }

        NodeList nodosToken = encabezado.getElementsByTagNameNS("*", "token");

        if (nodosToken.getLength() == 0) {
            throw new RuntimeException("Acceso denegado: No se encontró el token. Inicie sesión
primero.");
        }

        String token = nodosToken.item(0).getTextContent();

        if (!AdministradorTokens.validarToken(token)) {
            throw new RuntimeException("Acceso denegado: Token inválido o expirado.");
        }

      } catch (Exception e) {
        throw new RuntimeException(e.getMessage());
      }
    }
    return true;
  }

  @Override
  public Set<QName> getHeaders() {
    return null;
  }

  @Override
  public boolean handleFault(SOAPMessageContext contexto) {
    return true;
  }

  @Override
  public void close(MessageContext contexto) {}
}
```

4.1.12 Paso 12: Implementación de la Capa de Dominio y Conversión

La lógica de cálculo numérico se encapsula mediante la interfaz genérica IConversor<T> y sus implementaciones concretas:

- ConversorLongitud: Aplica factores de conversión respecto al metro (milímetro, centímetro, metro, kilómetro, yarda, pie, pulgada).
- ConversorMasa: Aplica factores de conversión normalizados al kilogramo (gramo, kilogramo, libra, onza, tonelada).
- ConversorTemperatura: Aplica fórmulas matemáticas directas e inversas para escalas Celsius, Fahrenheit y Kelvin.

Tabla 7
Interfaz Conversor (IConversor.java)

```
package ec.edu.monster.modelos.utilidades;

   public interface IConversor<T> {
      double convertir(double valor, T unidadInicial, T unidadFinal);
   }
```

Tabla 8
Conversor Longitud (ConversorLongitud.java)

```
package ec.edu.monster.modelos.utilidades;

import ec.edu.monster.modelos.utilidades.enums.UnidadLongitud;
import java.util.Map;

public class ConversorLongitud implements IConversor<UnidadLongitud> {

   private final Map<UnidadLongitud, Double> factores = Map.of(
      UnidadLongitud.MILIMETRO, 0.001,
      UnidadLongitud.CENTIMETRO, 0.01,
      UnidadLongitud.METRO, 1.0,
      UnidadLongitud.KILOMETRO, 1000.0,
      UnidadLongitud.YARDA, 0.9144
   );

   public ConversorLongitud() {}

   @Override
   public double convertir(double valor, UnidadLongitud unidadOrigen, UnidadLongitud
unidadFinal) {

      if (unidadOrigen == null || unidadFinal == null) {
         throw new IllegalArgumentException("Las unidades no pueden ser null");
      }
      if (!factores.containsKey(unidadOrigen) || !factores.containsKey(unidadFinal)) {
```

```
            throw new IllegalArgumentException("Unidad no soportada");
        }

        double valorEnBase = valor * factores.get(unidadOrigen);

        return valorEnBase / factores.get(unidadFinal);
    }
}
```

Tabla 9
Conversor Masa (ConversorMasa.java)

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package ec.edu.monster.modelos.utilidades;
import ec.edu.monster.modelos.utilidades.enums.UnidadMasa;
import java.util.Map;


/**
 *
 * @author MeatPuppets
 */
public class ConversorMasa implements IConversor<UnidadMasa> {

    private final Map<UnidadMasa, Double> factores = Map.of(
        UnidadMasa.MILIGRAMO, 0.001,
        UnidadMasa.GRAMO, 1.0,
        UnidadMasa.KILOGRAMO, 1000.0,
        UnidadMasa.TONELADA, 1_000_000.0,
        UnidadMasa.ONZA, 28.3495
    );

    public ConversorMasa() {}

    @Override
    public double convertir(double valor, UnidadMasa unidadOrigen, UnidadMasa unidadFinal) {

        if (unidadOrigen == null || unidadFinal == null) {
            throw new IllegalArgumentException("Las unidades no pueden ser null");
        }
        if (!factores.containsKey(unidadOrigen) || !factores.containsKey(unidadFinal)) {
            throw new IllegalArgumentException("Unidad no soportada");
```

```
        }

        double valorEnBase = valor * factores.get(unidadOrigen);

        return valorEnBase / factores.get(unidadFinal);
    }
}
```

Tabla 10
Conversor Temperatura (ConversorTemperatura.java)

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package ec.edu.monster.modelos.utilidades;

import ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura;
import static ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura.CELSIUS;
import static ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura.FAHRENHEIT;
import static ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura.KELVIN;
import static ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura.RANKINE;


/**
 *
 * @author MeatPuppets
 */
public class ConversorTemperatura implements IConversor<UnidadTemperatura> {

    public ConversorTemperatura() {}

    @Override
    public double convertir(double valor, UnidadTemperatura origen, UnidadTemperatura destino) {

        if (origen == null || destino == null) {
            throw new IllegalArgumentException("Las unidades no pueden ser null");
        }
        if (origen == destino) {
            return valor;
        }

        return switch (origen) {

            case CELSIUS -> switch (destino) {
                case FAHRENHEIT -> (valor * 9 / 5) + 32;
                case KELVIN -> valor + 273.15;
                case RANKINE -> (valor + 273.15) * 9 / 5;
```

```
            default -> throw new IllegalArgumentException("Conversión no soportada");
        };

        case FAHRENHEIT -> switch (destino) {
            case CELSIUS -> (valor - 32) * 5 / 9;
            case KELVIN -> (valor - 32) * 5 / 9 + 273.15;
            case RANKINE -> valor + 459.67;
            default -> throw new IllegalArgumentException("Conversión no soportada");
        };

        case KELVIN -> switch (destino) {
            case CELSIUS -> valor - 273.15;
            case FAHRENHEIT -> (valor - 273.15) * 9 / 5 + 32;
            case RANKINE -> valor * 9 / 5;
            default -> throw new IllegalArgumentException("Conversión no soportada");
        };

        case RANKINE -> switch (destino) {
            case CELSIUS -> (valor - 491.67) * 5 / 9;
            case FAHRENHEIT -> valor - 459.67;
            case KELVIN -> valor * 5 / 9;
            default -> throw new IllegalArgumentException("Conversión no soportada");
        };
    };
  }
}
```

Tabla 11
ServicioConversor.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package ec.edu.monster.servicios;
import ec.edu.monster.modelos.utilidades.IConversor;
/**
 *
 * @author MeatPuppets
 */
public class ServicioConversor<T> {

    private final IConversor<T> conversor;

    public ServicioConversor(IConversor<T> conversor) {
        this.conversor = conversor;
    }
```

```
    public double convertir(double valor, T origen, T destino) {
        if (conversor == null) {
            throw new IllegalStateException("No se ha inyectado un conversor");
        }
        return conversor.convertir(valor, origen, destino);
    }
}
```

Tabla 12
UnidadMapper.java

```
package ec.edu.monster.modelos.utilidades.mapeadores;

import ec.edu.monster.modelos.utilidades.enums.UnidadLongitud;
import ec.edu.monster.modelos.utilidades.enums.UnidadMasa;
import ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura;
import java.text.Normalizer;

/**
 * Mapeador para convertir cadenas de texto a los respectivos enumeradores de unidad.
 * Soporta nombres en mayúsculas/minúsculas, con/sin tildes, plurales y abreviaturas comunes.
 *
 * @author MeatPuppets
 */
public class UnidadMapper {

    private UnidadMapper() {
        // Constructor privado para clase utilitaria
    }

    private static String normalizar(String texto) {
        if (texto == null || texto.trim().isEmpty()) {
            throw new IllegalArgumentException("La unidad no puede estar vacía.");
        }
        String limpio = Normalizer.normalize(texto.trim(), Normalizer.Form.NFD)
                .replaceAll("\\p{M}", "")
                .toUpperCase();
        return limpio;
    }

    /**
     * Mapea una cadena de texto a UnidadLongitud.
     *
     * @param valor Cadena representativa de la unidad.
     * @return UnidadLongitud correspondiente.
     */
    public static UnidadLongitud toLongitud(String valor) {
        String u = normalizar(valor);
        return switch (u) {
            case "MILIMETRO", "MILIMETROS", "MM" -> UnidadLongitud.MILIMETRO;
```

```
            case "CENTIMETRO", "CENTIMETROS", "CM" -> UnidadLongitud.CENTIMETRO;
            case "METRO", "METROS", "M" -> UnidadLongitud.METRO;
            case "KILOMETRO", "KILOMETROS", "KM" -> UnidadLongitud.KILOMETRO;
            case "YARDA", "YARDAS", "YD" -> UnidadLongitud.YARDA;
            default -> {
                try {
                    yield UnidadLongitud.valueOf(u);
                } catch (IllegalArgumentException e) {
                    throw new IllegalArgumentException("Unidad de longitud no soportada: " + valor);
                }
            }
        };
    }

    /**
     * Mapea una cadena de texto a UnidadMasa.
     *
     * @param valor Cadena representativa de la unidad.
     * @return UnidadMasa correspondiente.
     */
    public static UnidadMasa toMasa(String valor) {
        String u = normalizar(valor);
        return switch (u) {
            case "MILIGRAMO", "MILIGRAMOS", "MG" -> UnidadMasa.MILIGRAMO;
            case "GRAMO", "GRAMOS", "G" -> UnidadMasa.GRAMO;
            case "KILOGRAMO", "KILOGRAMOS", "KG" -> UnidadMasa.KILOGRAMO;
            case "TONELADA", "TONELADAS", "T" -> UnidadMasa.TONELADA;
            case "ONZA", "ONZAS", "OZ" -> UnidadMasa.ONZA;
            default -> {
                try {
                    yield UnidadMasa.valueOf(u);
                } catch (IllegalArgumentException e) {
                    throw new IllegalArgumentException("Unidad de masa no soportada: " + valor);
                }
            }
        };
    }

    /**
     * Mapea una cadena de texto a UnidadTemperatura.
     *
     * @param valor Cadena representativa de la unidad.
     * @return UnidadTemperatura correspondiente.
     */
    public static UnidadTemperatura toTemperatura(String valor) {
        String u = normalizar(valor);
        return switch (u) {
            case "CELSIUS", "CENTIGRADO", "CENTIGRADOS", "C", "GRADOS CELSIUS" ->
UnidadTemperatura.CELSIUS;
            case "FAHRENHEIT", "F", "GRADOS FAHRENHEIT" -> UnidadTemperatura.FAHRENHEIT;
```

```
            case "KELVIN", "K", "GRADOS KELVIN" -> UnidadTemperatura.KELVIN;
            case "RANKINE", "R", "GRADOS RANKINE" -> UnidadTemperatura.RANKINE;
            default -> {
                try {
                    yield UnidadTemperatura.valueOf(u);
                } catch (IllegalArgumentException e) {
                    throw new IllegalArgumentException("Unidad de temperatura no soportada: " + valor);
                }
            }
        };
    }
}
```

Tabla 13
AdministradorCredenciales.java

```
package ec.edu.monster.seguridad;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Administrador de credenciales en memoria (sin base de datos).
 * Gestiona la autenticación y cambio de contraseñas de los usuarios.
 *
 * @author MeatPuppets
 */
public class AdministradorCredenciales {

    private static final Map<String, String> USUARIOS = new ConcurrentHashMap<>();

    // Credencial única permitida en el sistema
    public static final String USUARIO_DEFAULT = "MONSTER";
    public static final String CONTRASENIA_DEFAULT = "MONSTER9";

    static {
        reiniciar();
    }

    private AdministradorCredenciales() {
        // Constructor privado para clase utilitaria
    }

    /**
     * Restablece las credenciales a la única credencial permitida:
     * USUARIO: MONSTER
     * PASSWORD: MONSTER9
     */
    public static void reiniciar() {
```

```
        USUARIOS.clear();
        USUARIOS.put(USUARIO_DEFAULT.toLowerCase(), CONTRASENIA_DEFAULT);
    }

    /**
     * Valida si el usuario y la contraseña coinciden.
     *
     * @param usuario     Nombre de usuario.
     * @param contrasenia Contraseña ingresada.
     * @return true si las credenciales son válidas, false en caso contrario.
     */
    public static boolean validarUsuario(String usuario, String contrasenia) {
        if (usuario == null || contrasenia == null) {
            return false;
        }
        String claveAlmacenada = USUARIOS.get(usuario.trim().toLowerCase());
        return claveAlmacenada != null && claveAlmacenada.equals(contrasenia);
    }

    /**
     * Cambia la contraseña buscando el usuario cuya contraseña actual coincide con
     * la provista.
     * Si no se encuentra coincidencia, lanza RuntimeException.
     *
     * @param contraseniaActual Contraseña actual del usuario.
     * @param contraseniaNueva  Nueva contraseña a establecer.
     */
    public static void cambiarContrasenia(String contraseniaActual, String contraseniaNueva) {
        if (contraseniaActual == null || contraseniaNueva == null ||
contraseniaNueva.trim().isEmpty()) {
            throw new RuntimeException("Las contraseñas no pueden ser nulas ni vacías.");
        }

        boolean actualizada = false;
        for (Map.Entry<String, String> entry : USUARIOS.entrySet()) {
            if (entry.getValue().equals(contraseniaActual)) {
                USUARIOS.put(entry.getKey(), contraseniaNueva);
                actualizada = true;
                break;
            }
        }

        if (!actualizada) {
            throw new RuntimeException("La contraseña actual es incorrecta.");
        }
    }

    /**
     * Cambia la contraseña para un usuario específico.
     *
```

```
     * @param usuario         Nombre de usuario.
     * @param contraseniaActual Contraseña actual.
     * @param contraseniaNueva  Nueva contraseña.
     */
    public static void cambiarContrasenia(String usuario, String contraseniaActual, String
contraseniaNueva) {
        if (usuario == null || contraseniaActual == null || contraseniaNueva == null
                || contraseniaNueva.trim().isEmpty()) {
            throw new RuntimeException("Datos inválidos para cambio de contraseña.");
        }
        String userKey = usuario.trim().toLowerCase();
        String claveActual = USUARIOS.get(userKey);
        if (claveActual == null || !claveActual.equals(contraseniaActual)) {
            throw new RuntimeException("Credenciales incorrectas para el usuario: " + usuario);
        }
        USUARIOS.put(userKey, contraseniaNueva);
    }

    /**
     * Registra o actualiza un usuario en memoria.
     *
     * @param usuario     Nombre de usuario.
     * @param contrasenia Contraseña del usuario.
     */
    public static void registrarUsuario(String usuario, String contrasenia) {
        if (usuario != null && contrasenia != null && !usuario.isBlank() && !contrasenia.isBlank()) {
            USUARIOS.put(usuario.trim().toLowerCase(), contrasenia);
        }
    }
}
```

Tabla 14
AdministradorTokens.java

```
package ec.edu.monster.seguridad;

/**
 * Adaptador/Alias para AdministradorToken para compatibilidad con
ManejadorAutenticacion.
 *
 * @author MeatPuppets
 */
public class AdministradorTokens {

    private AdministradorTokens() {
        // Constructor privado para clase utilitaria
    }

    public static String generarToken() {
        return AdministradorToken.generarToken();
```

```
    }

    public static boolean validarToken(String token) {
        return AdministradorToken.validarToken(token);
    }

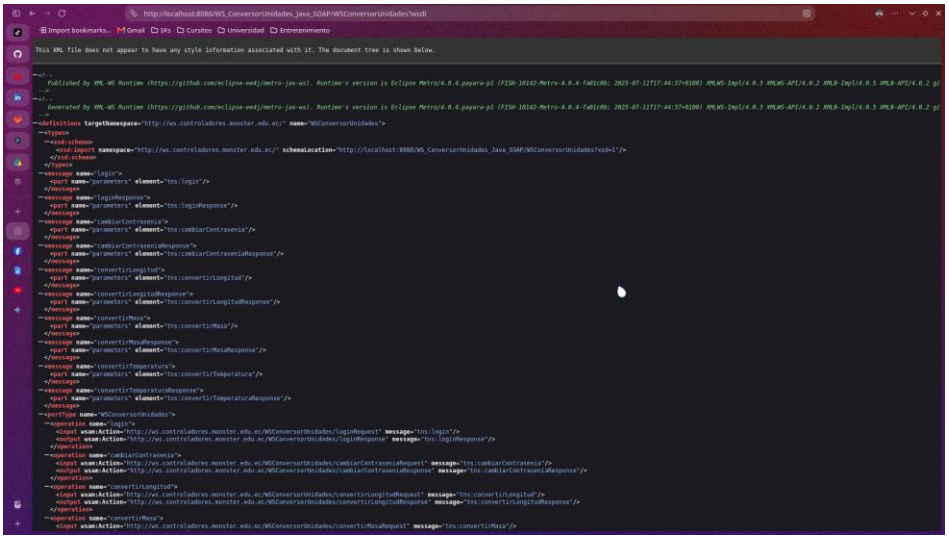
    public static void invalidarToken(String token) {
        AdministradorToken.invalidarToken(token);
    }

    public static void limpiar() {
        AdministradorToken.limpiar();
    }
}
```

4.1.13 Paso 13: Despliegue en Servidor y Comprobación del WSDL

Se compila la aplicación y se despliega sobre Payara Server. Se verifica la correcta exposición del contrato accediendo a la URL del descriptor en el navegador web: **http://localhost:8080/WS_ConversorUnidades_Java_SOAP/WSConversorUnidades?wsdl**

Figura 11
Contrato WSDL publicado en Payara Server.

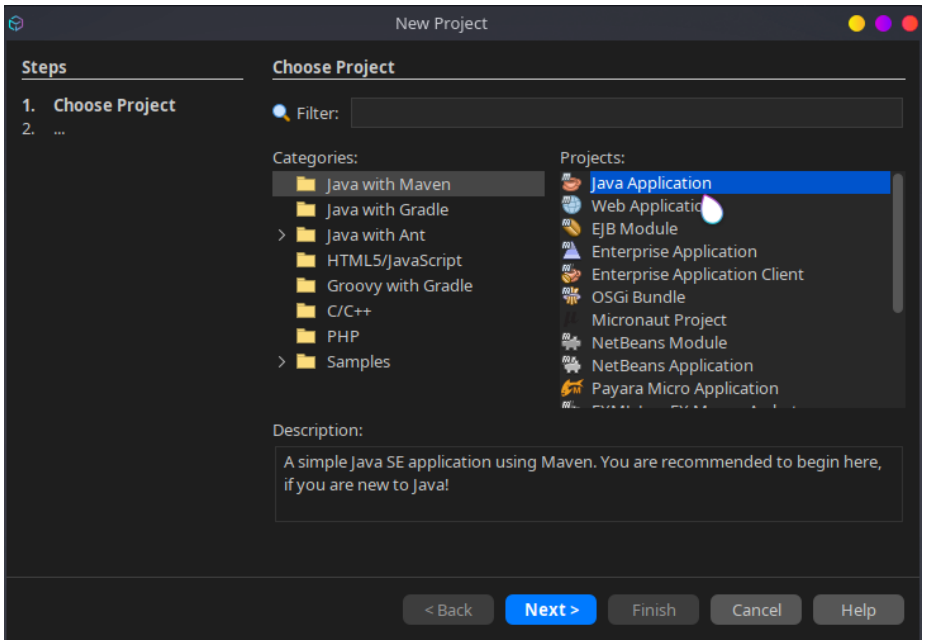


4.2 Cliente Consola

4.2.1 Paso 1: Creación del Proyecto Maven en NetBeans

En el entorno NetBeans IDE, se crea un nuevo proyecto seleccionando Java with Maven > Java Application. Se asigna el nombre WSConversorConsoleClient, se establece la ruta física de almacenamiento y se parametriza el GroupId como ec.edu.monster.

Figura 12
Creación del proyecto Maven para el cliente consola en Netbeans.



4.2.2 Paso 2: Consumo del WSDL mediante el Asistente Gráfico Web Service Client

Para integrar el servicio web SOAP de forma estandarizada y evitar la definición manual de interfaces o contratos, se utiliza el asistente automatizado de NetBeans:

1. Se hace clic derecho sobre el nodo del proyecto WSConversorConsoleClient y se selecciona New > Other > Web Services > Web Service Client (o directamente New > Web Service Client...).
2. En la opción WSDL URL, se introduce la dirección física del contrato publicado por Payara Server: http://localhost:8080/WS_ConversorUnidades_Java_SOAP/WSConversorUnidades?wsdl
3. Se especifica el paquete de destino para los artefactos generados como ec.edu.monster.ws.
4. Al presionar Finish, NetBeans configura automáticamente en el archivo pom.xml el plugin jaxws-maven-plugin con el objetivo wsimport dentro de la fase generate-sources, crea el descriptor de mapeo src/jax-ws-catalog.xml y descarga las definiciones de esquema para compilar los proxies de cliente:

a. WSConversorUnidades_Service: Localizador del servicio y fábrica de puertos SOAP.
b. WSConversorUnidades: Interfaz proxy fuertemente tipada que expone las operaciones remotas (login, cambiarContrasenia, convertirLongitud, convertirMasa, convertirTemperatura).

Figura 13
Asistente Web Service Client para consumo de WSDL en NetBeans.

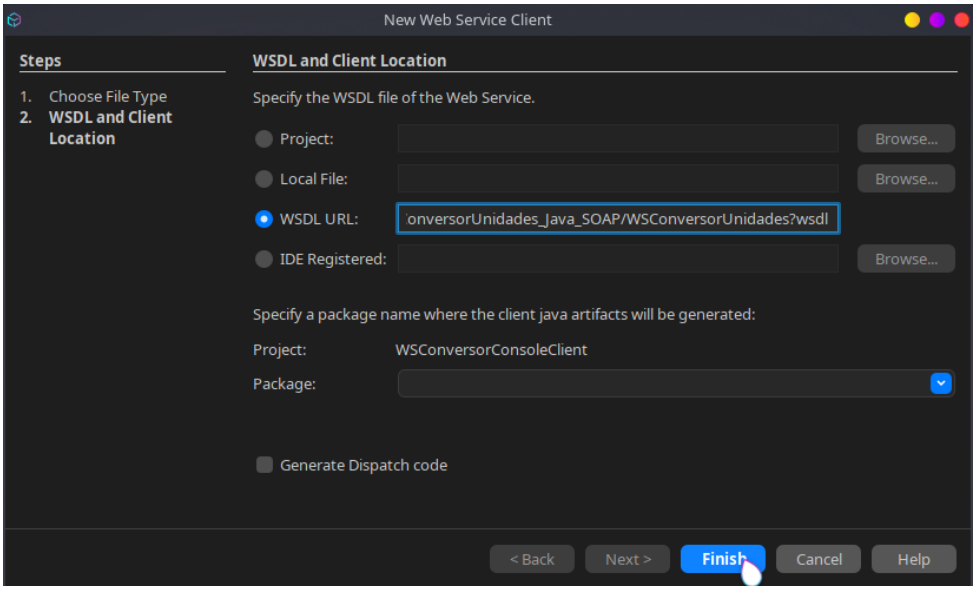
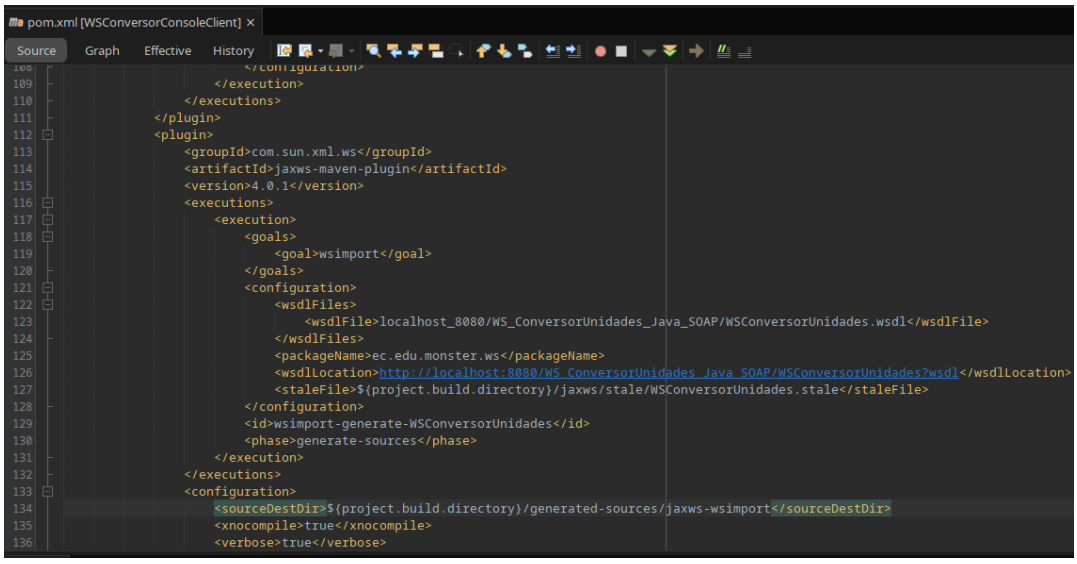


Figura 14
Configuración de jaxws-maven-plugin y referencias WSDL generadas

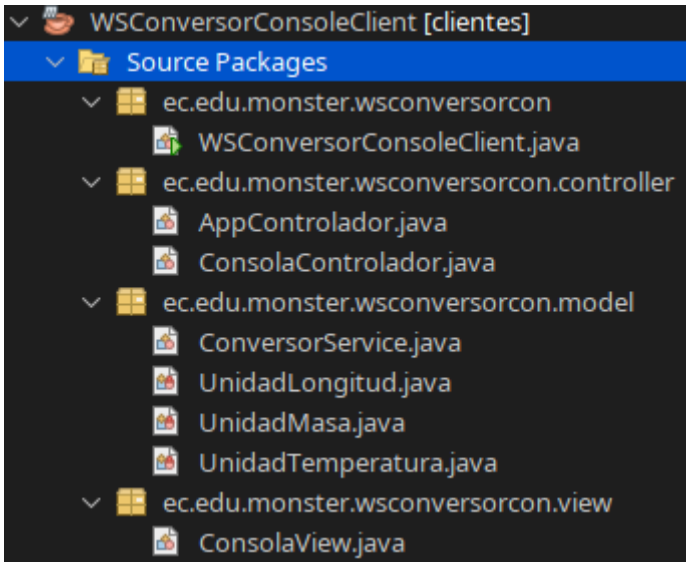


4.2.3 Paso 3: Estructuración Modular de Paquetes (MVC)

Se organiza el código fuente en src/main/java bajo el patrón arquitectónico Modelo-Vista-Controlador:

- ec.edu.monster.wsconversorcon.model: Contiene las clases de datos y el servicio ConversorService.
- ec.edu.monster.wsconversorcon.view: Contiene la clase ConsolaView para la interfaz de texto.
- ec.edu.monster.wsconversorcon.controller: Aloja AppControlador y ConsolaControlador.

Figura 15
Estructura de paquetes MVC del Cliente Consola en NetBeans



4.2.4 Paso 4: Implementación de la Capa de Modelo (ConversorService)

La clase ConversorService encapsula la lógica de comunicación con los proxies generados:

1. Instancia el servicio WSConversorUnidades_Service y obtiene la interfaz operativa ConversorUnidades vinculada al endpoint.
2. Tras una autenticación exitosa mediante login(usuario, contrasenia), almacena en memoria el token criptográfico retornado.
3. Para la ejecución de operaciones de conversión, inyecta programáticamente el token de seguridad dentro de los encabezados SOAP de la petición utilizando un manejador de protocolo o configurando los metadatos de transporte en el contexto de la llamada (BindingProvider).

Tabla *15*
ConversorService.java

```
package ec.edu.monster.wsconversorcon.model;

import ec.edu.monster.ws.WSConversorUnidades;
import ec.edu.monster.ws.WSConversorUnidades_Service;
```

```
public class ConversorService {

    private WSConversorUnidades getPort() {
        WSConversorUnidades_Service service = new WSConversorUnidades_Service();
        return service.getWSConversorUnidadesPort();
    }

    public String login(String usuario, String contrasenia) {
        return getPort().login(usuario, contrasenia);
    }

    public double convertirLongitud(double valor, String unidadInicial, String unidadFinal) {
        return getPort().convertirLongitud(valor, unidadInicial, unidadFinal);
    }

    public double convertirMasa(double valor, String unidadInicial, String unidadFinal) {
        return getPort().convertirMasa(valor, unidadInicial, unidadFinal);
    }

    public double convertirTemperatura(double valor, String unidadInicial, String unidadFinal)
{
        return getPort().convertirTemperatura(valor, unidadInicial, unidadFinal);
    }
}
```

4.2.5 Paso 5: Implementación de Controladores y Vista de Consola

1. ConsolaView: Provee métodos de presentación en terminal que imprimen menús interactivos, solicitan credenciales de acceso, capturan valores numéricos con validación de tipo y despliegan los resultados calculados.

Tabla 16
ConsolaView.java

```
package ec.edu.monster.wsconversorcon.view;

import ec.edu.monster.wsconversorcon.model.UnidadLongitud;
import ec.edu.monster.wsconversorcon.model.UnidadMasa;
import ec.edu.monster.wsconversorcon.model.UnidadTemperatura;

import java.util.Scanner;

public class ConsolaView {

    private final Scanner scanner = new Scanner(System.in);

    public void mostrarBanner() {
```

```
    System.out.println("   /\\_______/\\");
    System.out.println("  /   o   o   \\
╔═══════════════════════════════════════════════════════════════════════════╗");
    System.out.println(" (   == ^ ==   )   ╟   MONSTERS INC. - CONVERSOR SOAP      ╟");
    System.out.println("  )   \\___/   (    ╟   (Arquitectura MVC - Java Puro)      ╟");
    System.out.println(" (  (_______)  )   ╟   Personaje: James P. Sullivan        ╟");
    System.out.println("  \\_____________/
╚═══════════════════════════════════════════════════════════════════════════╝");
    System.out.println();
  }

  public String leerTexto(String etiqueta) {
    System.out.print(etiqueta + ": ");
    return scanner.nextLine().trim();
  }

  public String leerContrasenia(String etiqueta) {
    if (System.console() != null) {
      char[] pass = System.console().readPassword(etiqueta + ": ");
      return new String(pass);
    } else {
      return leerTexto(etiqueta);
    }
  }

  public double leerDouble(String etiqueta) {
    while (true) {
      System.out.print(etiqueta + ": ");
      String input = scanner.nextLine().trim();
      try {
        return Double.parseDouble(input);
      } catch (NumberFormatException e) {
        System.out.println(" [!] Por favor ingrese un número válido.");
      }
    }
  }

  public int leerEntero(String etiqueta, int min, int max) {
    while (true) {
      System.out.print(etiqueta + " [" + min + "-" + max + "]: ");
      String input = scanner.nextLine().trim();
      try {
        int valor = Integer.parseInt(input);
        if (valor >= min && valor <= max) {
          return valor;
        }
        System.out.println(" [!] La opción debe estar entre " + min + " y " + max + ".");
      } catch (NumberFormatException e) {
        System.out.println(" [!] Ingrese un número entero válido.");
      }
```

```
    }
}

public void mostrarMenuPrincipal() {
    System.out.println();
    System.out.println("--- MENÚ PRINCIPAL ---");
    System.out.println("1. Conversor de Longitudes");
    System.out.println("2. Conversor de Masa");
    System.out.println("3. Conversor de Temperatura");
    System.out.println("4. Salir");
}

public String seleccionarUnidadLongitud(String momento) {
    System.out.println("\nSeleccione Unidad " + momento + ":");
    UnidadLongitud[] valores = UnidadLongitud.values();
    for (int i = 0; i < valores.length; i++) {
        System.out.println((i + 1) + ". " + valores[i].name() + " (" + valores[i].getEtiqueta() + ")");
    }
    int seleccion = leerEntero("Opción", 1, valores.length);
    return valores[seleccion - 1].name();
}

public String seleccionarUnidadMasa(String momento) {
    System.out.println("\nSeleccione Unidad " + momento + ":");
    UnidadMasa[] valores = UnidadMasa.values();
    for (int i = 0; i < valores.length; i++) {
        System.out.println((i + 1) + ". " + valores[i].name() + " (" + valores[i].getEtiqueta() + ")");
    }
    int seleccion = leerEntero("Opción", 1, valores.length);
    return valores[seleccion - 1].name();
}

public String seleccionarUnidadTemperatura(String momento) {
    System.out.println("\nSeleccione Unidad " + momento + ":");
    UnidadTemperatura[] valores = UnidadTemperatura.values();
    for (int i = 0; i < valores.length; i++) {
        System.out.println((i + 1) + ". " + valores[i].name() + " (" + valores[i].getEtiqueta() + ")");
    }
    int seleccion = leerEntero("Opción", 1, valores.length);
    return valores[seleccion - 1].name();
}

public void mostrarResultado(double resultado) {
    System.out.println();
    System.out.println("================================================");
    System.out.println("Resultado: " + resultado);
    System.out.println("================================================");
}

public void mostrarMensaje(String mensaje) {
```

```
        System.out.println(" > " + mensaje);
    }

    public void mostrarError(String error) {
        System.err.println(" [X] Error: " + error);
    }
}
```

2. ConsolaControlador: Administra el bucle de ejecución de la interfaz CLI, evalúa las opciones ingresadas por el operador y delega la ejecución al AppControlador, asegurando que no se permita la conversión de magnitudes sin un inicio de sesión previo.

Tabla 17
ConsolaControlador.java

```
package ec.edu.monster.wsconversorcon.controller;

import ec.edu.monster.wsconversorcon.view.ConsolaView;

public class ConsolaControlador {

    private final ConsolaView vista = new ConsolaView();
    private final AppControlador appControlador = new AppControlador();

    public void iniciar() {
        vista.mostrarBanner();

        if (!realizarLogin()) {
            vista.mostrarMensaje("Acceso cancelado. Finalizando programa.");
            return;
        }

        bucleMenuPrincipal();
    }

    private boolean realizarLogin() {
        System.out.println("\n--- INICIO DE SESIÓN ---");
        System.out.println("(Credenciales oficiales: MONSTER / MONSTER9)");

        while (true) {
            String usuario = vista.leerTexto("Usuario");
            String contrasenia = vista.leerContrasenia("Contraseña");

            if (usuario.isEmpty() || contrasenia.isEmpty()) {
                vista.mostrarMensaje("Por favor complete todos los campos.");
```

```
            continue;
        }

        try {
            String token = appControlador.login(usuario, contrasenia);
            if (token != null && !token.isBlank()) {
                vista.mostrarMensaje("¡Bienvenido " + usuario + "! Sesión iniciada correctamente.");
                return true;
            } else {
                vista.mostrarError("Usuario o contraseña incorrectos.");
            }
        } catch (Exception e) {
            vista.mostrarError("No se pudo conectar con el servidor SOAP o credenciales incorrectas:
" + e.getMessage());
        }

        String reintentar = vista.leerTexto("¿Desea reintentar? (s/n)");
        if (!reintentar.equalsIgnoreCase("s")) {
            return false;
        }
    }
  }

  private void bucleMenuPrincipal() {
    boolean continuar = true;

    while (continuar) {
      vista.mostrarMenuPrincipal();
      int opcion = vista.leerEntero("Seleccione una opción", 1, 4);

      switch (opcion) {
        case 1:
          procesarLongitud();
          break;
        case 2:
          procesarMasa();
          break;
        case 3:
          procesarTemperatura();
          break;
        case 4:
          continuar = false;
          vista.mostrarMensaje("¡Gracias por utilizar el WS Conversor! Hasta luego.");
          break;
      }
    }
  }

  private void procesarLongitud() {
    try {
```

```
            System.out.println("\n>>> CONVERSOR DE LONGITUDES <<<");
            String unidadInicial = vista.seleccionarUnidadLongitud("Inicial");
            double valor = vista.leerDouble("Ingrese el valor a convertir");
            String unidadFinal = vista.seleccionarUnidadLongitud("Final");

            double resultado = appControlador.convertirLongitud(valor, unidadInicial, unidadFinal);
            vista.mostrarResultado(resultado);
        } catch (Exception e) {
            vista.mostrarError("Error en la conversión: " + e.getMessage());
        }
    }

    private void procesarMasa() {
        try {
            System.out.println("\n>>> CONVERSOR DE MASA <<<");
            String unidadInicial = vista.seleccionarUnidadMasa("Inicial");
            double valor = vista.leerDouble("Ingrese el valor a convertir");
            String unidadFinal = vista.seleccionarUnidadMasa("Final");

            double resultado = appControlador.convertirMasa(valor, unidadInicial, unidadFinal);
            vista.mostrarResultado(resultado);
        } catch (Exception e) {
            vista.mostrarError("Error en la conversión: " + e.getMessage());
        }
    }

    private void procesarTemperatura() {
        try {
            System.out.println("\n>>> CONVERSOR DE TEMPERATURA <<<");
            String unidadInicial = vista.seleccionarUnidadTemperatura("Inicial");
            double valor = vista.leerDouble("Ingrese el valor a convertir");
            String unidadFinal = vista.seleccionarUnidadTemperatura("Final");

            double resultado = appControlador.convertirTemperatura(valor, unidadInicial, unidadFinal);
            vista.mostrarResultado(resultado);
        } catch (Exception e) {
            vista.mostrarError("Error en la conversión: " + e.getMessage());
        }
    }
}
```

Tabla 18
AppControlador.java

```
package ec.edu.monster.wsconversorcon.controller;

import ec.edu.monster.wsconversorcon.model.ConversorService;

public class AppControlador {
```

```
    private final ConversorService service = new ConversorService();

    public String login(String usuario, String contrasenia) {
        return service.login(usuario, contrasenia);
    }

    public double convertirLongitud(double valor, String unidadInicial, String unidadFinal) {
        return service.convertirLongitud(valor, unidadInicial, unidadFinal);
    }

    public double convertirMasa(double valor, String unidadInicial, String unidadFinal) {
        return service.convertirMasa(valor, unidadInicial, unidadFinal);
    }

    public double convertirTemperatura(double valor, String unidadInicial, String unidadFinal) {
        return service.convertirTemperatura(valor, unidadInicial, unidadFinal);
    }
}
```

Tabla 19
WSConversorConsoleClient.java

```
package ec.edu.monster.wsconversorcon;

import ec.edu.monster.wsconversorcon.controller.ConsolaControlador;

public class WSConversorConsoleClient {

    public static void main(String[] args) {
        ConsolaControlador controlador = new ConsolaControlador();
        controlador.iniciar();
    }
}
```

Figura 16
Ejecución del Cliente Consola (Inicio de sesión en terminal)

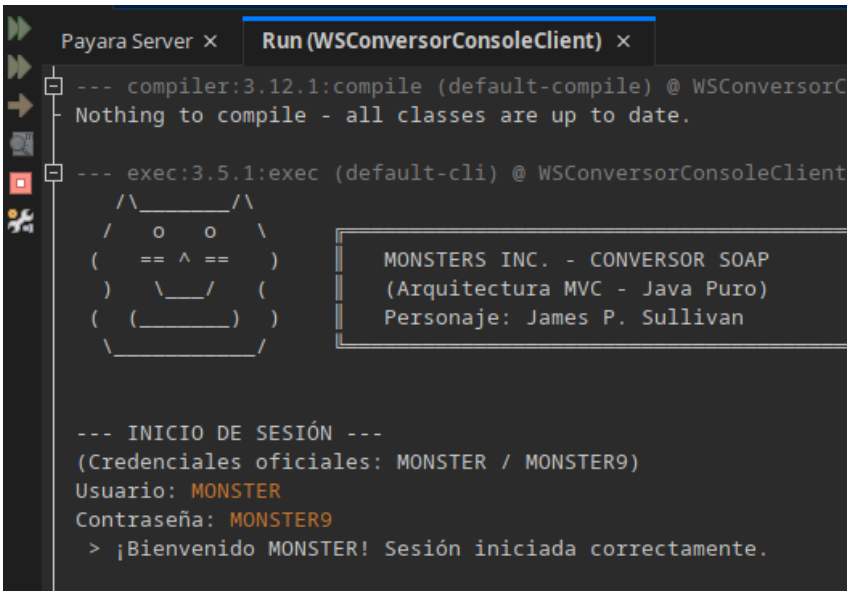
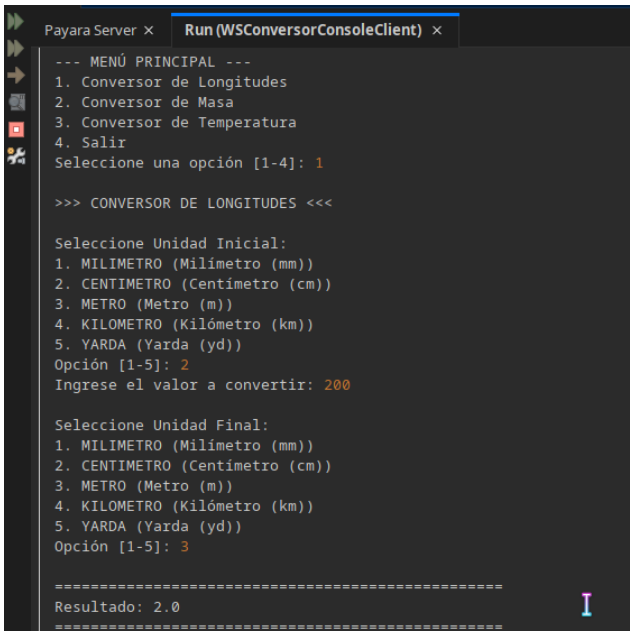


Figura 17
Ejecución de operaciones de conversión en Cliente Consola

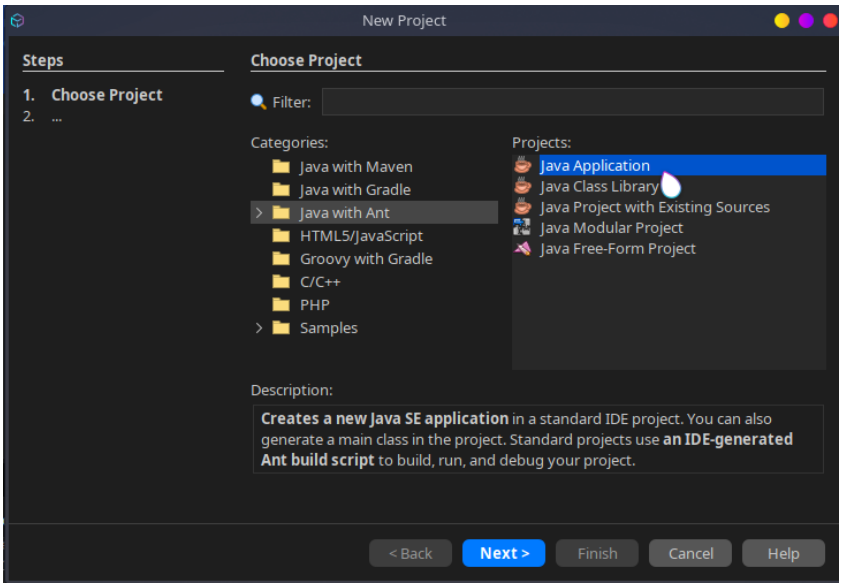


4.3 Cliente Escritorio

4.3.1 Paso 1: Creación del Proyecto Gráfico en NetBeans

Se crea un proyecto de tipo Java with Ant > Java Application en NetBeans IDE denominado WSConversorClient, aprovechando el soporte nativo del gestor Ant para la integración visual de componentes Swing y la gestión de clientes de servicios web.

Figura 18
Creación del Proyecto Java Swing en NetBeans



4.3.2 Paso 2: Generación de Clientes Proxy mediante el Asistente Web Service Client

Para enlazar el cliente de escritorio con el servicio web alojado en Payara Server:

1. Se hace clic derecho sobre el proyecto WSConversorClient y se selecciona New > Web Service Client…
2. En la opción WSDL URL, se ingresa la URL de publicación del servidor: http://localhost:8080/WS_ConversorUnidades_Java_SOAP/WSConversorUnidades?wsdl
3. Se asigna como paquete de destino ec.edu.monster.ws.
4. NetBeans ejecuta internamente la tarea Ant wsimport, creando:
    a. El archivo de metadatos nbproject/jax-ws.xml y el script de compilación nbproject/jaxws-build.xml.
    b. El catálogo local de resolución src/META-INF/jax-ws-catalog.xml.
    c. El repositorio de esquemas XML en xml-resources/web-service-references/WSConversorUnidades/.
    d. Las clases cliente generadas en src/ec/edu/monster/ws: WSConversorUnidades_Service.java y WSConversorUnidades.java.

Figura 19
Asistente Web Service Client en Cliente Escritorio

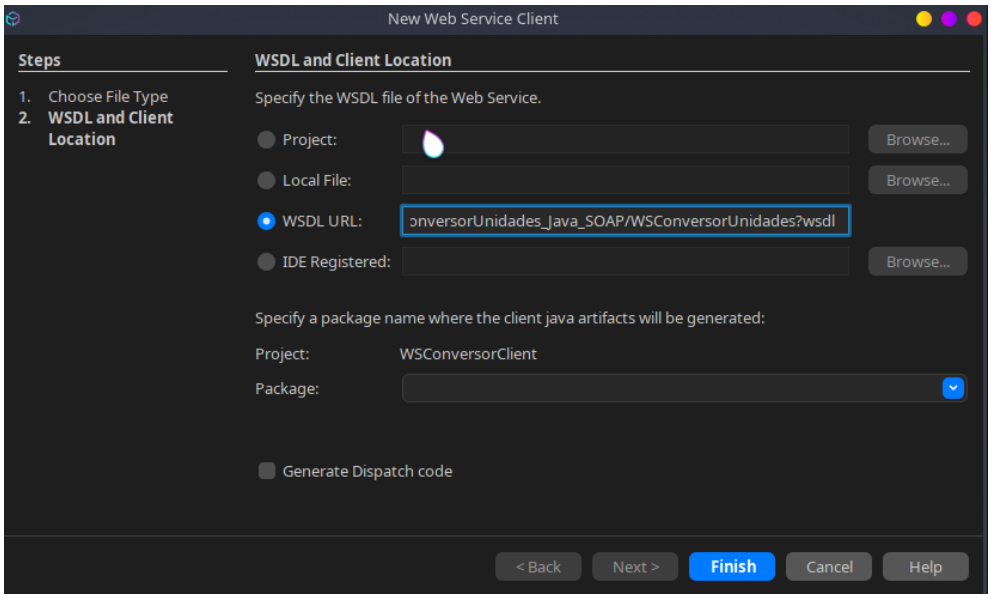
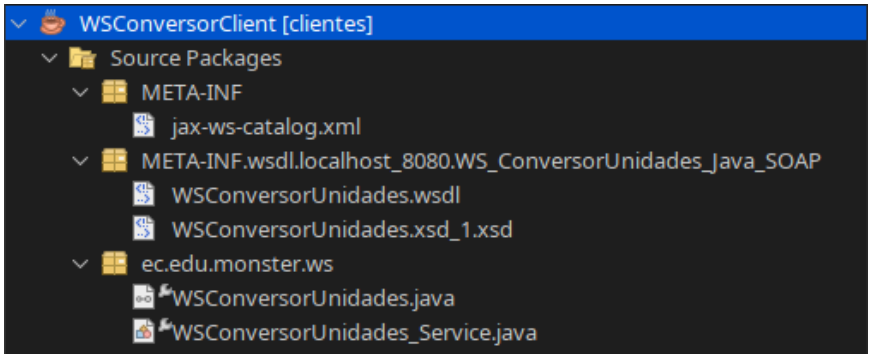


Figura 20
Artefactos generados en xml-resources y paquetes de stubs SOAP



4.3.3 Paso 3: Estructuración de Paquetes MVC

Se organizan los paquetes en el directorio src:

- ec.edu.monster.wsconversorclient.model: Entidades de magnitud (UnidadLongitud, UnidadMasa, UnidadTemperatura) y el servicio de consumo ConversorService.
- ec.edu.monster.wsconversorclient.view: Formularios gráficos LoginView y ConversorView.
- ec.edu.monster.wsconversorclient.controller: Controlador de eventos de interfaz AppControlador.

4.3.4 Paso 4: Implementación de la Capa de Modelo (ConversorService)

Se implementa ConversorService, clase responsable de instanciar WSConversorUnidades_Service, obtener el puerto de comunicación e inyectar el token de autenticación en los encabezados de las peticiones SOAP tras el inicio de sesión.

Tabla 20
ConversorService.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package ec.edu.monster.wsconversorclient.model;

import ec.edu.monster.ws.WSConversorUnidades;
import ec.edu.monster.ws.WSConversorUnidades_Service;

/**
 *
 * @author meatpuppets
 */
public class ConversorService {

    private static String tokenSesion;

    public static String getTokenSesion() {
        return tokenSesion;
    }

    public static void setTokenSesion(String token) {
        tokenSesion = token;
    }

    public String login(String usuario, String contrasenia) {
        WSConversorUnidades_Service service = new WSConversorUnidades_Service();
        WSConversorUnidades port = service.getWSConversorUnidadesPort();
        String token = port.login(usuario, contrasenia);
        tokenSesion = token;
        return token;
    }

    public double convertirLongitud(double valor, String unidadInicial, String unidadFinal) {
        WSConversorUnidades_Service service = new WSConversorUnidades_Service();
        WSConversorUnidades port = service.getWSConversorUnidadesPort();
        return port.convertirLongitud(valor, unidadInicial, unidadFinal);
    }
```

```
    public double convertirMasa(double valor, String unidadInicial, String unidadFinal) {
        WSConversorUnidades_Service service = new WSConversorUnidades_Service();
        WSConversorUnidades port = service.getWSConversorUnidadesPort();
        return port.convertirMasa(valor, unidadInicial, unidadFinal);
    }

    public double convertirTemperatura(double valor, String unidadInicial, String unidadFinal) {
        WSConversorUnidades_Service service = new WSConversorUnidades_Service();
        WSConversorUnidades port = service.getWSConversorUnidadesPort();
        return port.convertirTemperatura(valor, unidadInicial, unidadFinal);
    }

}
```

4.3.5 Paso 5: Diseño de Vistas con Formularios Swing

Utilizando el diseñador gráfico de NetBeans (Matisse GUI Builder), se construyen las ventanas de usuario:

- LoginView.java: Formulario modal basado en JFrame provisto de campos JTextField para el usuario, JPasswordField para la contraseña y un botón de autenticación.
- ConversorView.java: Ventana principal del aplicativo que integra selectores desplegables JComboBox para la magnitud física, la unidad origen y la unidad destino, una caja de texto para el valor de entrada y un área no editable para el resultado del cálculo.

Figura 21
Diseño visual del formulario de login en Swing (LoginView)

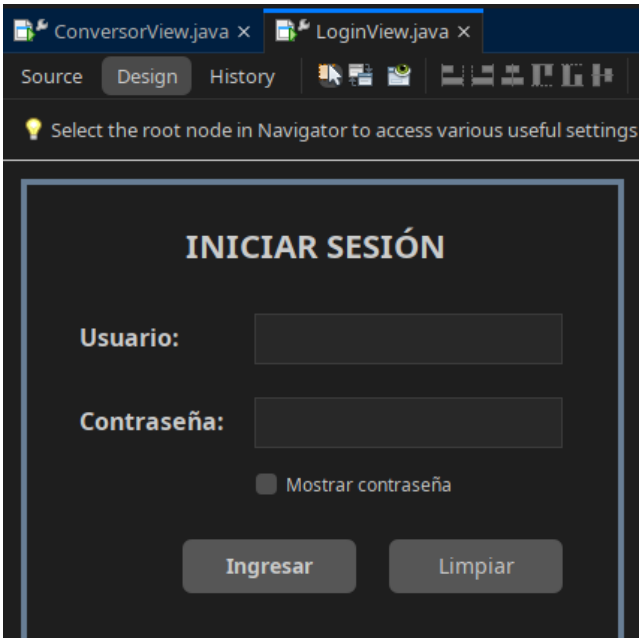
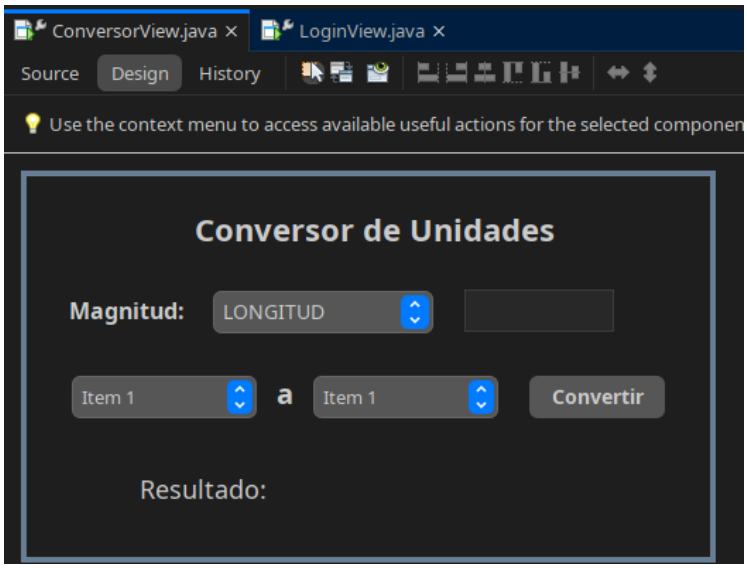


Figura 22
Diseño visual de la ventana principal del conversor (ConversorView)



4.3.6 Paso 6: Implementación del Controlador de Eventos (AppControlador)

La clase AppControlador implementa la interfaz ActionListener para gestionar las acciones sobre los botones del entorno gráfico. Captura los eventos de clic, extrae los parámetros de la vista, invoca de manera síncrona los métodos de ConversorService, gestiona posibles excepciones protocolarias (SOAPFaultException) y actualiza los componentes gráficos o despliega mensajes mediante JOptionPane.

Tabla 21
AppControlador.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package ec.edu.monster.wsconversorclient.controller;

import ec.edu.monster.wsconversorclient.model.ConversorService;

/**
 *
 * @author meatpuppets
 */
public class AppControlador {

    private final ConversorService service = new ConversorService();

    public String login(String usuario, String contrasenia) {
```

```
        return service.login(usuario, contrasenia);
    }

    public double convertirLongitud(double n1, String unidadInicial, String unidadFinal) {
        return service.convertirLongitud(n1, unidadInicial, unidadFinal);
    }

    public double convertirLogintud(double n1, String unidadInicial, String unidadFinal) {
        return service.convertirLongitud(n1, unidadInicial, unidadFinal);
    }

    public double convertirMasa(double n1, String unidadInicial, String unidadFinal){
        return service.convertirMasa(n1, unidadInicial, unidadFinal);
    }

    public double convertirTemperatura(double n1, String unidadInicial, String unidadFinal){
        return service.convertirTemperatura(n1, unidadInicial, unidadFinal);
    }
}
```

Figura 23
Interfaz gráfica de Login ejecutada en Cliente Escritorio



Figura 24
Operación de conversión exitosa en Cliente Escritorio



4.4 Cliente Móvil

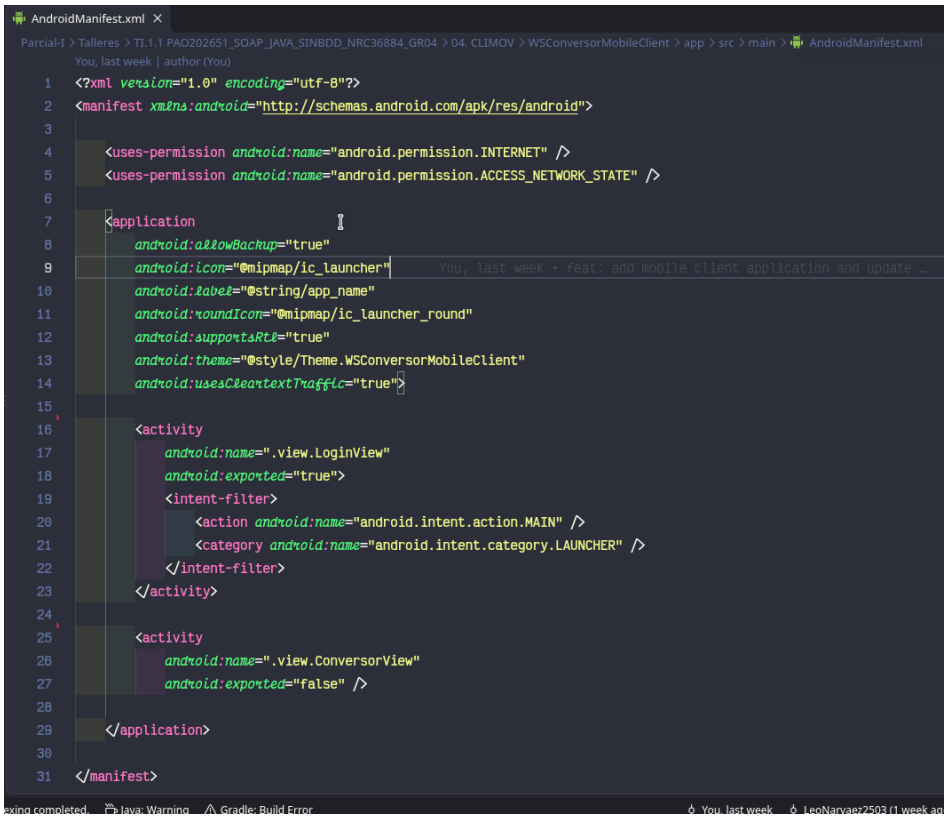
4.4.1 Paso 1: Creación y Configuración del Proyecto Android en Gradle

Se genera un proyecto móvil nativo en Android Studio utilizando el lenguaje de programación Java. En el archivo de manifiesto AndroidManifest.xml, se configuran los permisos indispensables para la comunicación a través de sockets HTTP:

<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />

Asimismo, en la etiqueta <application> se habilita el atributo android:usesCleartextTraffic="true" para permitir el tráfico HTTP no cifrado hacia el entorno local de desarrollo.

Figura
Configuración de permisos de red en AndroidManifest.xml



4.4.2 Paso 2: Estructuración Modular bajo el Patrón MVC

Dentro del módulo app/src/main/java, se estructuran los paquetes de la aplicación:

- ec.edu.monster.wsconversormov.model: Clases de soporte, gestión de sesión (UsuarioSession), configuración de red (ConfiguracionConexion) y ConversorService.
- ec.edu.monster.wsconversormov.view: Actividades de interfaz gráfica (LoginView y ConversorView).
- ec.edu.monster.wsconversormov.controller: Controlador de eventos móviles AppControlador.

Figura 26
Estructura de paquetes MVC



4.4.3 Paso 3: Implementación de la Capa de Modelo y Despacho SOAP Asíncrono

Dado que el entorno móvil Android prohíbe estrictamente la ejecución de operaciones de red en el hilo principal (UI Thread) lanzando una excepción NetworkOnMainThreadException, se implementa ConversorService bajo un esquema de procesamiento en segundo plano:

1. Se emplea un ejecutor de subprocesos (ExecutorService) para desacoplar el tráfico de red de la interfaz visual.
2. ConfiguracionConexion resuelve la dirección IP del servidor Payara (utilizando el alias 10.0.2.2 para el emulador de Android o la dirección física de la red local para terminales reales).
3. Se construyen los paquetes XML con la envoltura SOAP Envelope, incorporando el elemento <auth:token> en el encabezado SOAP Header y despachando las solicitudes mediante conexiones HttpURLConnection hacia el endpoint del servidor.
4. Se parsea la respuesta XML recibida y se notifica el resultado al hilo de interfaz de usuario mediante runOnUiThread().

Tabla *22*
Modelo de comuniación SOAP asíncrona móvil.

```
package ec.edu.monster.wsconversormov.model;

import ec.edu.monster.wsconversormov.model.ConfiguracionConexion;

import java.io.BufferedReader;
import java.io.InputStream;
```

```
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class ConversorService {

   private static final String TARGET_NAMESPACE = "http://ws.controladores.monster.edu.ec/";

   public String login(String usuario, String contrasenia) throws Exception {
      String soapBody = String.format(
         "<ws:login xmlns:ws=\"%s\">" +
         "<usuario>%s</usuario>" +
         "<contrasenia>%s</contrasenia>" +
         "</ws:login>",
         TARGET_NAMESPACE, escapeXml(usuario), escapeXml(contrasenia)
      );

      return invocarSoap(soapBody);
   }

   public double convertirLongitud(double valor, String unidadInicial, String unidadFinal) throws
Exception {
      String soapBody = String.format(
         "<ws:convertirLongitud xmlns:ws=\"%s\">" +
         "<valor>%s</valor>" +
         "<unidadInicial>%s</unidadInicial>" +
         "<unidadFinal>%s</unidadFinal>" +
         "</ws:convertirLongitud>",
         TARGET_NAMESPACE, valor, escapeXml(unidadInicial), escapeXml(unidadFinal)
      );

      String respuesta = invocarSoap(soapBody);
      return Double.parseDouble(respuesta.trim());
   }

   public double convertirMasa(double valor, String unidadInicial, String unidadFinal) throws
Exception {
      String soapBody = String.format(
         "<ws:convertirMasa xmlns:ws=\"%s\">" +
         "<valor>%s</valor>" +
         "<unidadInicial>%s</unidadInicial>" +
         "<unidadFinal>%s</unidadFinal>" +
         "</ws:convertirMasa>",
         TARGET_NAMESPACE, valor, escapeXml(unidadInicial), escapeXml(unidadFinal)
      );
```

```
        String respuesta = invocarSoap(soapBody);
        return Double.parseDouble(respuesta.trim());
    }

    public double convertirTemperatura(double valor, String unidadInicial, String unidadFinal)
throws Exception {
        String soapBody = String.format(
            "<ws:convertirTemperatura xmlns:ws=\"%s\">" +
            "<valor>%s</valor>" +
            "<unidadInicial>%s</unidadInicial>" +
            "<unidadFinal>%s</unidadFinal>" +
            "</ws:convertirTemperatura>",
            TARGET_NAMESPACE, valor, escapeXml(unidadInicial), escapeXml(unidadFinal)
        );

        String respuesta = invocarSoap(soapBody);
        return Double.parseDouble(respuesta.trim());
    }

    private String invocarSoap(String cuerpoXml) throws Exception {
        String soapEnvelope = "<?xml version=\"1.0\" encoding=\"utf-8\"?>" +
                "<soapenv:Envelope xmlns:soapenv=\"http://schemas.xmlsoap.org/soap/envelope/\">"
+
                "<soapenv:Header/>" +
                "<soapenv:Body>" +
                cuerpoXml +
                "</soapenv:Body>" +
                "</soapenv:Envelope>";

        java.net.URL url = java.net.URI.create(ConfiguracionConexion.getEndpointUrl()).toURL();
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("POST");
        conn.setConnectTimeout(8000);
        conn.setReadTimeout(8000);
        conn.setRequestProperty("Content-Type", "text/xml; charset=utf-8");
        conn.setRequestProperty("SOAPAction", "\"\"");
        conn.setDoOutput(true);

        try (OutputStream os = conn.getOutputStream()) {
            byte[] input = soapEnvelope.getBytes(StandardCharsets.UTF_8);
            os.write(input, 0, input.length);
        }

        int statusCode = conn.getResponseCode();
        InputStream is = (statusCode >= 200 && statusCode < 400) ? conn.getInputStream() :
conn.getErrorStream();

        StringBuilder response = new StringBuilder();
        if (is != null) {
            try (BufferedReader br = new BufferedReader(new InputStreamReader(is,
```

```
StandardCharsets.UTF_8))) {
            String line;
            while ((line = br.readLine()) != null) {
               response.append(line);
            }
         }
      }

      String rawXml = response.toString();

      if (statusCode >= 400 || rawXml.contains("faultstring") || rawXml.contains("Fault")) {
         String fault = extraerEtiqueta(rawXml, "faultstring");
         if (fault == null || fault.isEmpty()) {
            fault = "Error en el servidor SOAP (HTTP " + statusCode + ").";
         }
         throw new RuntimeException(fault);
      }

      String resultado = extraerEtiqueta(rawXml, "return");
      if (resultado == null) {
         throw new RuntimeException("Respuesta vacía del servidor SOAP.");
      }

      return resultado;
   }

   private String extraerEtiqueta(String xml, String tagName) {
      Pattern pattern = Pattern.compile("<(?:\\w+:)?(" + tagName + ")[^>]*>(.*?)</(?:\\w+:)?\\1>",
Pattern.DOTALL);
      Matcher matcher = pattern.matcher(xml);
      if (matcher.find()) {
         return matcher.group(2).trim();
      }
      return null;
   }

   private String escapeXml(String str) {
      if (str == null) return "";
      return str.replace("&", "&amp;")
            .replace("<", "&lt;")
            .replace(">", "&gt;")
            .replace("\"", "&quot;")
            .replace("'", "&apos;");
   }
}
```

4.4.4 Paso 4: Diseño de Layouts XML

En el directorio res/layout, se diseñan las interfaces gráficas:

- **activity_login.xml:** Layout lineal con campos EditText para credenciales y un botón Button estilizado para el envío.
- **activity_conversor.xml:** Vista principal con selectores Spinner para la elección interactiva de magnitudes y unidades, un campo de texto para la cantidad a convertir y un contenedor de texto para el resultado.

Figura 27
Diseño visual de layout de login

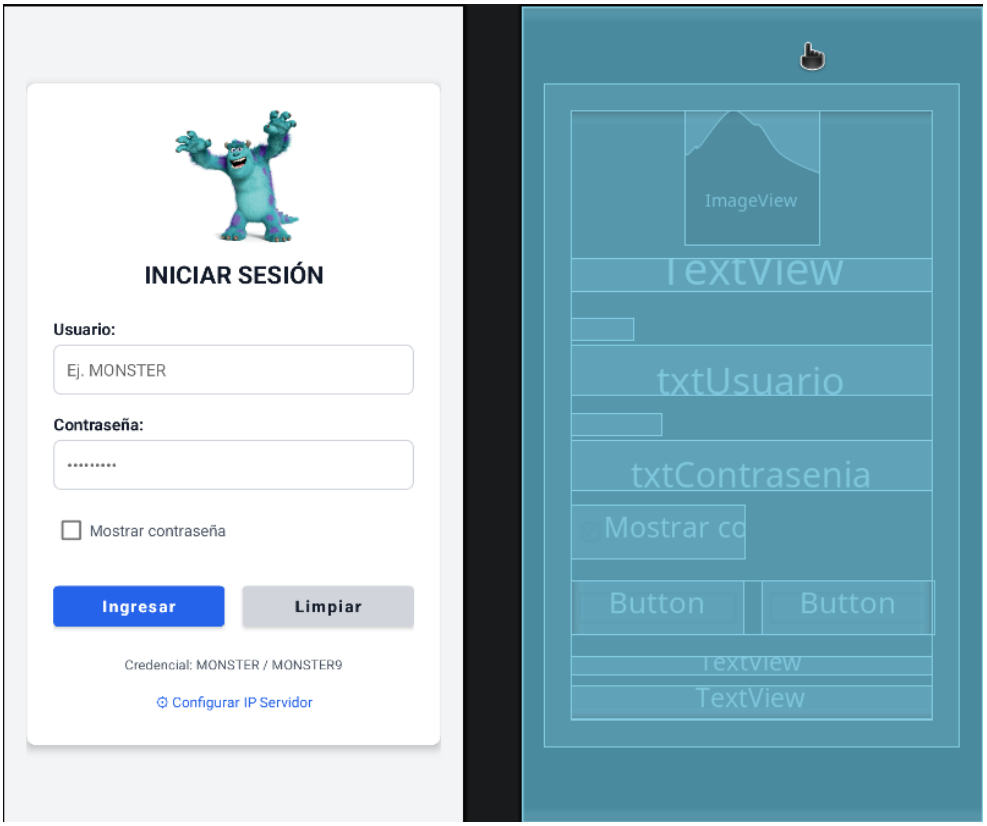


Figura 28
Diseño visual de layout del conversor

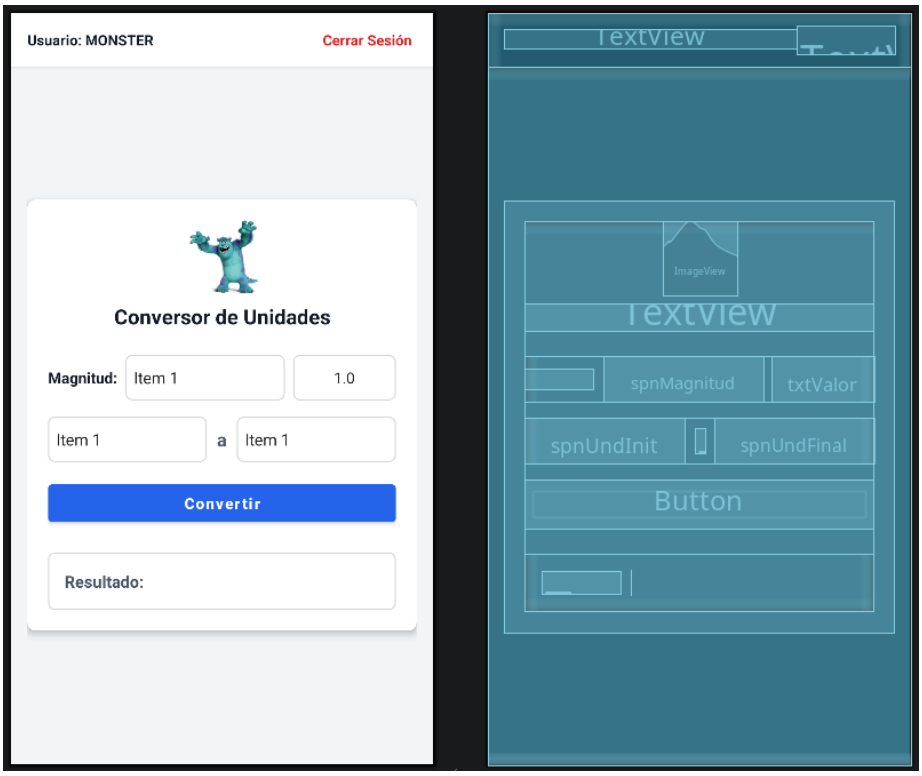


Tabla 23
Diseño XML de la vista de autenticación móvil.

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/background"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:orientation="vertical"
        android:padding="20dp">

        <!-- Card estilo ventana sencilla -->
        <androidx.cardview.widget.CardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            app:cardBackgroundColor="@color/card_bg"
            app:cardCornerRadius="8dp"
            app:cardElevation="4dp"
```

```
app:contentPadding="24dp">

<LinearLayout
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:orientation="vertical">

   <!-- Logo Sulley Monsters Inc -->
   <ImageView
      android:id="@+id/imgSulley"
      android:layout_width="120dp"
      android:layout_height="120dp"
      android:layout_gravity="center_horizontal"
      android:layout_marginBottom="12dp"
      android:contentDescription="@string/app_name"
      android:src="@drawable/sullivan" />

   <TextView
      android:id="@+id/lblTituloLogin"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:layout_marginBottom="24dp"
      android:gravity="center"
      android:text="@string/title_login"
      android:textColor="@color/text_primary"
      android:textSize="22sp"
      android:textStyle="bold" />

   <!-- Campo Usuario -->
   <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_marginBottom="4dp"
      android:text="@string/prompt_usuario"
      android:textColor="@color/text_primary"
      android:textSize="15sp"
      android:textStyle="bold" />

   <EditText
      android:id="@+id/txtUsuario"
      android:layout_width="match_parent"
      android:layout_height="45dp"
      android:layout_marginBottom="16dp"
      android:background="@drawable/edittext_border"
      android:hint="Ej. MONSTER"
      android:inputType="text"
      android:paddingHorizontal="12dp"
      android:textSize="15sp" />

   <!-- Campo Contraseña -->
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginBottom="4dp"
    android:text="@string/prompt_contrasenia"
    android:textColor="@color/text_primary"
    android:textSize="15sp"
    android:textStyle="bold" />

<EditText
    android:id="@+id/txtContrasenia"
    android:layout_width="match_parent"
    android:layout_height="45dp"
    android:layout_marginBottom="12dp"
    android:background="@drawable/edittext_border"
    android:hint="•••••••••"
    android:inputType="textPassword"
    android:paddingHorizontal="12dp"
    android:textSize="15sp" />

<!-- Checkbox Mostrar contraseña -->
<CheckBox
    android:id="@+id/chkMostrarContrasenia"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginBottom="20dp"
    android:text="@string/chk_mostrar_contrasenia"
    android:textColor="@color/text_secondary"
    android:textSize="14sp" />

<!-- Botones Ingresar y Limpiar -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center"
    android:orientation="horizontal">

    <Button
        android:id="@+id/btnIngresar"
        android:layout_width="0dp"
        android:layout_height="48dp"
        android:layout_marginEnd="8dp"
        android:layout_weight="1"
        android:backgroundTint="@color/primary"
        android:text="@string/btn_ingresar"
        android:textAllCaps="false"
        android:textColor="@color/card_bg"
        android:textSize="15sp"
        android:textStyle="bold" />
```

```
            <Button
                android:id="@+id/btnLimpiar"
                android:layout_width="0dp"
                android:layout_height="48dp"
                android:layout_marginStart="8dp"
                android:layout_weight="1"
                android:backgroundTint="@color/border"
                android:text="@string/btn_limpiar"
                android:textAllCaps="false"
                android:textColor="@color/text_primary"
                android:textSize="15sp"
                android:textStyle="bold" />
        </LinearLayout>

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="20dp"
            android:gravity="center"
            android:text="Credencial: MONSTER / MONSTER9"
            android:textColor="@color/text_secondary"
            android:textSize="12sp" />

        <!-- Botón para configurar IP del servidor -->
        <TextView
            android:id="@+id/btnConfigIp"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="10dp"
            android:gravity="center"
            android:padding="6dp"
            android:text="@string/btn_config_ip"
            android:textColor="@color/primary"
            android:textSize="13sp" />

      </LinearLayout>
    </androidx.cardview.widget.CardView>

  </LinearLayout>
</ScrollView>
```

Tabla 24

Diseño XML de la vista principal móvil

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
```

```
android:background="@color/background"
android:orientation="vertical">

<!-- Barra superior simple: Usuario y Cerrar Sesión -->
<LinearLayout
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:background="@color/card_bg"
   android:elevation="2dp"
   android:gravity="center_vertical"
   android:orientation="horizontal"
   android:paddingHorizontal="16dp"
   android:paddingVertical="12dp">

   <TextView
      android:id="@+id/txtUsuarioSesion"
      android:layout_width="0dp"
      android:layout_height="wrap_content"
      android:layout_weight="1"
      android:text="Usuario: MONSTER"
      android:textColor="@color/text_primary"
      android:textSize="14sp"
      android:textStyle="bold" />

   <TextView
      android:id="@+id/btnCerrarSesion"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:padding="4dp"
      android:text="@string/btn_cerrar_sesion"
      android:textColor="@color/danger"
      android:textSize="14sp"
      android:textStyle="bold" />
</LinearLayout>

<ScrollView
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:fillViewport="true">

   <LinearLayout
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:gravity="center"
      android:orientation="vertical"
      android:padding="16dp">

      <!-- Card tipo ventana de escritorio -->
      <androidx.cardview.widget.CardView
         android:layout_width="match_parent"
```

```
android:layout_height="wrap_content"
app:cardBackgroundColor="@color/card_bg"
app:cardCornerRadius="8dp"
app:cardElevation="4dp"
app:contentPadding="20dp">

<LinearLayout
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:orientation="vertical">

  <!-- Logo Sulley Monsters Inc -->
  <ImageView
    android:id="@+id/imgSulleyConversor"
    android:layout_width="72dp"
    android:layout_height="72dp"
    android:layout_gravity="center_horizontal"
    android:layout_marginBottom="8dp"
    android:contentDescription="@string/app_name"
    android:src="@drawable/sullivan" />

  <!-- Título dinámico -->
  <TextView
    android:id="@+id/txtTitulo"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="24dp"
    android:gravity="center"
    android:text="@string/title_conversor"
    android:textColor="@color/text_primary"
    android:textSize="20sp"
    android:textStyle="bold" />

  <!-- Fila 1: Magnitud y Valor -->
  <LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="16dp"
    android:gravity="center_vertical"
    android:orientation="horizontal">

    <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_marginEnd="8dp"
      android:text="@string/prompt_magnitud"
      android:textColor="@color/text_primary"
      android:textSize="15sp"
      android:textStyle="bold" />
```

```
    <Spinner
        android:id="@+id/spnMagnitud"
        android:layout_width="0dp"
        android:layout_height="44dp"
        android:layout_marginEnd="8dp"
        android:layout_weight="1"
        android:background="@drawable/edittext_border" />

    <EditText
        android:id="@+id/txtValor"
        android:layout_width="100dp"
        android:layout_height="44dp"
        android:background="@drawable/edittext_border"
        android:gravity="center"
        android:hint="1.0"
        android:inputType="numberDecimal"
        android:paddingHorizontal="8dp"
        android:text="1.0"
        android:textSize="15sp" />
</LinearLayout>

<!-- Fila 2: undInit  a  undFinal -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="16dp"
    android:gravity="center_vertical"
    android:orientation="horizontal">

    <Spinner
        android:id="@+id/spnUndInit"
        android:layout_width="0dp"
        android:layout_height="44dp"
        android:layout_weight="1"
        android:background="@drawable/edittext_border" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginHorizontal="10dp"
        android:text="@string/prompt_a"
        android:textColor="@color/text_secondary"
        android:textSize="18sp"
        android:textStyle="bold" />

    <Spinner
        android:id="@+id/spnUndFinal"
        android:layout_width="0dp"
        android:layout_height="44dp"
        android:layout_weight="1"
```

```
            android:background="@drawable/edittext_border" />
        </LinearLayout>

        <!-- Botón Convertir -->
        <Button
          android:id="@+id/btnConvertir"
          android:layout_width="match_parent"
          android:layout_height="48dp"
          android:layout_marginBottom="24dp"
          android:backgroundTint="@color/primary"
          android:text="@string/btn_convertir"
          android:textAllCaps="false"
          android:textColor="@color/card_bg"
          android:textSize="15sp"
          android:textStyle="bold" />

        <!-- Fila 3: Resultado: [ conversionResultado ] -->
        <LinearLayout
          android:layout_width="match_parent"
          android:layout_height="wrap_content"
          android:background="@drawable/edittext_border"
          android:gravity="center_vertical"
          android:orientation="horizontal"
          android:padding="16dp">

          <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginEnd="10dp"
            android:text="@string/label_resultado"
            android:textColor="@color/text_secondary"
            android:textSize="16sp"
            android:textStyle="bold" />

          <TextView
            android:id="@+id/txtResultado"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text=""
            android:textColor="@color/accent"
            android:textSize="18sp"
            android:textStyle="bold" />
        </LinearLayout>

      </LinearLayout>
    </androidx.cardview.widget.CardView>

  </LinearLayout>
</ScrollView>
```

```
</LinearLayout>
```

4.4.5 Paso 5: Implementación de Actividades y Ejecución Móvil

Las clases LoginView y ConversorView extienden de AppCompatActivity e inicializan los controladores de interfaz en el método onCreate(), vinculando las acciones del usuario con AppControlador.

Tabla *25*

Controlador de actividades móviles

```
package ec.edu.monster.wsconversormov.controller;

import ec.edu.monster.wsconversormov.model.ConversorService;

public class AppControlador {

   private final ConversorService service = new ConversorService();

   public String login(String usuario, String contrasenia) throws Exception {
      return service.login(usuario, contrasenia);
   }

   public double convertirLongitud(double valor, String unidadInicial, String unidadFinal) throws
Exception {
      return service.convertirLongitud(valor, unidadInicial, unidadFinal);
   }

   public double convertirMasa(double valor, String unidadInicial, String unidadFinal) throws
Exception {
      return service.convertirMasa(valor, unidadInicial, unidadFinal);
   }

   public double convertirTemperatura(double valor, String unidadInicial, String unidadFinal)
throws Exception {
      return service.convertirTemperatura(valor, unidadInicial, unidadFinal);
   }
}
```

Figura 29
Ejecución del Cliente Móvil (Login)

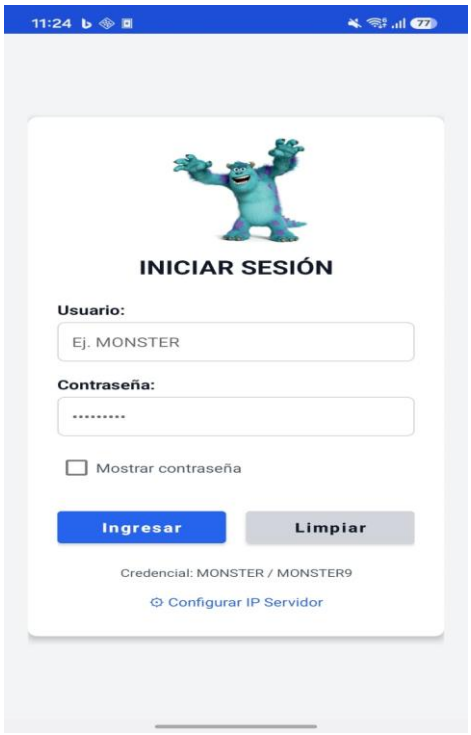
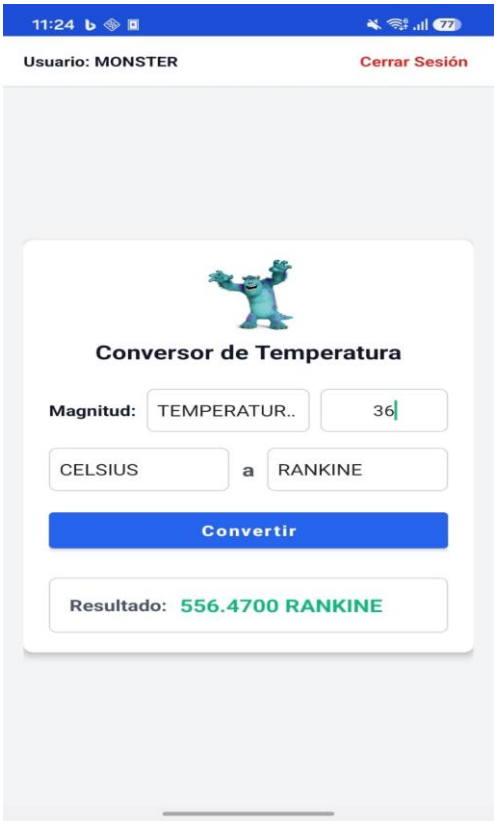


Figura 30
Ejecución del Cliente Móvil (Conversor)

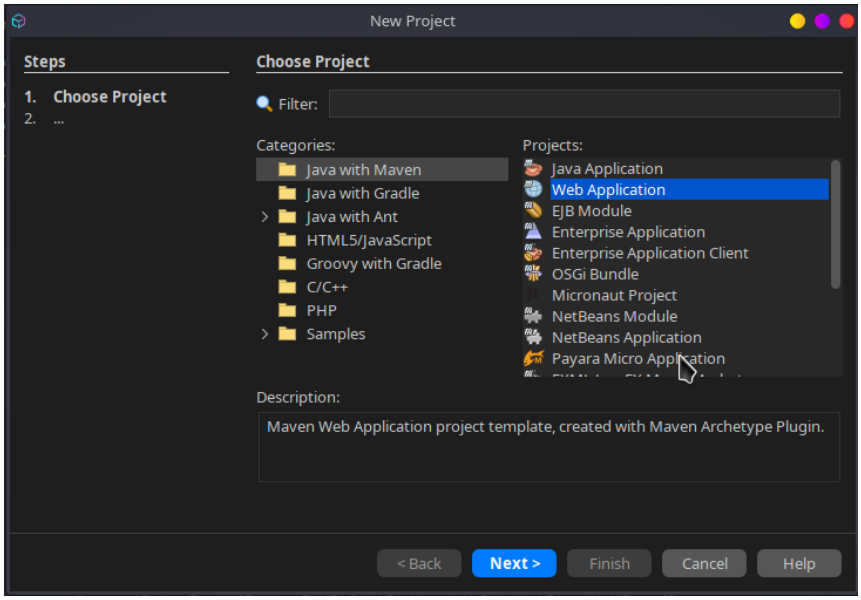


4.5 Cliente Web

4.5.1 Paso 1: Creación del Proyecto Web Jakarta Faces en NetBeans

Se crea un proyecto en NetBeans IDE clasificado como Java with Maven > Web Application denominado WSConversorWebClient, configurando el soporte de tecnologías JavaServer Faces (Jakarta Faces) sobre el contenedor Payara Server.

Figura 31
Creación del proyecto Web Jakarta Face en Netbeans



4.5.2 Paso 2: Configuración del Entorno JSF y Descriptores

En el directorio WEB-INF, se estructuran los archivos de despliegue y control:

- web.xml: Mapea el servlet principal de Faces (FacesServlet) con el patrón de URL *.xhtml para interceptar las peticiones del ciclo de vida de JSF.
- beans.xml: Habilita la inyección de dependencias CDI con modo de descubrimiento annotated.
- faces-config.xml: Declara la activación del framework Jakarta Faces.

Figura 32
Descriptores de despliegue web.xml, beans.xml y faces-config.xml

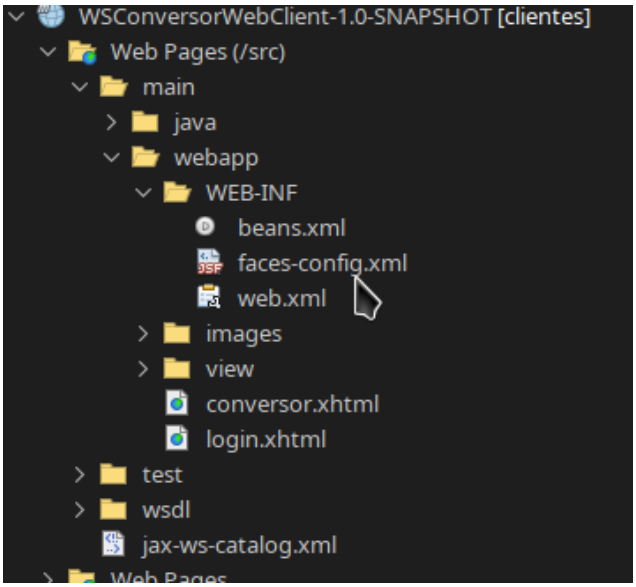


Tabla 26
Descriptores de despliegue JSF

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
        version="6.0">

    <display-name>WSConversorWebClient</display-name>

    <!-- Jakarta Faces Servlet -->
    <servlet>
        <servlet-name>Faces Servlet</servlet-name>
        <servlet-class>jakarta.faces.webapp.FacesServlet</servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
        <servlet-name>Faces Servlet</servlet-name>
        <url-pattern>*.xhtml</url-pattern>
    </servlet-mapping>

    <welcome-file-list>
        <welcome-file>login.xhtml</welcome-file>
    </welcome-file-list>

    <session-config>
        <session-timeout>30</session-timeout>
    </session-config>
```

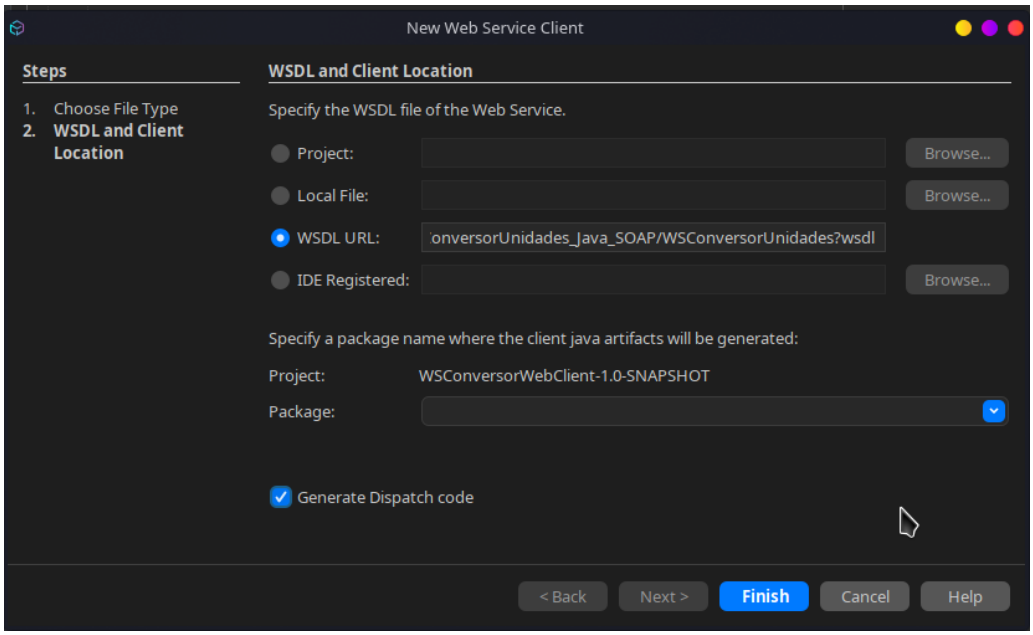
```
</web-app>
```

4.5.3 Paso 3: Consumo del WSDL mediante el Asistente Web Service Client

Al igual que en el cliente de consola, se aprovecha el asistente gráfico de NetBeans en el proyecto web:

1. Se hace clic derecho sobre WSConversorWebClient y se selecciona New > Web Service Client...
2. Se introduce la URL del WSDL: http://localhost:8080/WS_ConversorUnidades_Java_SOAP/WSConversorUnidades?wsdl
3. Se selecciona como paquete de destino ec.edu.monster.ws.
4. NetBeans incorpora en el pom.xml del cliente web la ejecución del plugin jaxws-maven-plugin y genera el catálogo de resolución src/jax-ws-catalog.xml, proveyendo acceso directo a los proxies cliente dentro de los Managed Beans de la aplicación.

Figura 33
Asistente Web Service Client en Proyecto Web



4.5.4 Paso 4: Implementación de la Capa de Modelo (ConversorService)

La clase ConversorService administra las llamadas al puerto proxy WSConversorUnidades, inyectando el token de sesión en las cabeceras SOAP para cada petición autorizada. Asimismo, la clase UsuarioSession mantiene el estado de la sesión activa en el ámbito del usuario HTTP.

Tabla 27
Modelo de consumo SOAP en Cliente Web

```
package ec.edu.monster.wsconversorweb.model;

import ec.edu.monster.ws.WSConversorUnidades;
import ec.edu.monster.ws.WSConversorUnidades_Service;

public class ConversorService {

    private WSConversorUnidades getPort() {
        WSConversorUnidades_Service service = new WSConversorUnidades_Service();
        return service.getWSConversorUnidadesPort();
    }

    public String login(String usuario, String contrasenia) {
        return getPort().login(usuario, contrasenia);
    }

    public double convertirLongitud(double valor, String unidadInicial, String unidadFinal) {
        return getPort().convertirLongitud(valor, unidadInicial, unidadFinal);
    }

    public double convertirMasa(double valor, String unidadInicial, String unidadFinal) {
        return getPort().convertirMasa(valor, unidadInicial, unidadFinal);
    }

    public double convertirTemperatura(double valor, String unidadInicial, String unidadFinal) {
        return getPort().convertirTemperatura(valor, unidadInicial, unidadFinal);
    }

    public String cambiarContrasenia(String actual, String nueva) {
        return getPort().cambiarContrasenia(actual, nueva);
    }
}
```

4.5.5 Paso 5: Implementación de Managed Beans Controladores con CDI

Se implementan controladores mediante anotaciones estándar de Jakarta Faces y CDI:

- LoginView (@Named, @SessionScoped): Captura las credenciales ingresadas, delega la autenticación a ConversorService, preserva el token en la sesión HTTP y ejecuta la redirección hacia la vista del conversor (faces-redirect=true).

- ConversorView (@Named, @ViewScoped): Mantiene el estado de la vista interactiva, gestiona las listas de unidades disponibles actualizándolas según la magnitud seleccionada y procesa la petición de cálculo.

Tabla 28

Managed Bean de vista reactiva

```
package ec.edu.monster.wsconversorweb.view;
import ec.edu.monster.wsconversorweb.controller.AppControlador;
import ec.edu.monster.wsconversorweb.model.HistorialItem;
import ec.edu.monster.wsconversorweb.model.UnidadLongitud;
import ec.edu.monster.wsconversorweb.model.UnidadMasa;
import ec.edu.monster.wsconversorweb.model.UnidadTemperatura;
import jakarta.annotation.PostConstruct;
import jakarta.enterprise.context.SessionScoped;
import jakarta.faces.application.FacesMessage;
import jakarta.faces.context.FacesContext;
import jakarta.inject.Inject;
import jakarta.inject.Named;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * Vista / Bean del Conversor de Unidades para Cliente Web (Patrón MVC).
 */
@Named("conversorView")
@SessionScoped
public class ConversorView implements Serializable {

   private static final long serialVersionUID = 1L;

   @Inject
   private LoginView loginView;

   private String tipoMagnitud = "LONGITUD";
   private Double valor;
   private String unidadOrigen;
   private String unidadDestino;
   private Double resultado;
   private final List<HistorialItem> historial = new ArrayList<>();
   private final AppControlador controlador = new AppControlador();

   @PostConstruct
   public void init() {
      actualizarUnidades();
   }

   public void verificarSesion() {
      if (loginView == null || !loginView.isAutenticado()) {
```

```
            try {
                FacesContext.getCurrentInstance().getExternalContext().redirect("login.xhtml");
            } catch (Exception ignored) {
            }
        }
    }

    public void cambiarMagnitud() {
        actualizarUnidades();
        resultado = null;
    }

    private void actualizarUnidades() {
        List<String> disponibles = getUnidadesDisponibles();
        if (!disponibles.isEmpty()) {
            unidadOrigen = disponibles.get(0);
            unidadDestino = disponibles.size() > 1 ? disponibles.get(1) : disponibles.get(0);
        }
    }

    public List<String> getUnidadesDisponibles() {
        switch (tipoMagnitud) {
            case "MASA":
                return Arrays.stream(UnidadMasa.values()).map(Enum::name).toList();
            case "TEMPERATURA":
                return Arrays.stream(UnidadTemperatura.values()).map(Enum::name).toList();
            case "LONGITUD":
            default:
                return Arrays.stream(UnidadLongitud.values()).map(Enum::name).toList();
        }
    }

    public String getTitulo() {
        switch (tipoMagnitud) {
            case "MASA":
                return "Conversor de Masa";
            case "TEMPERATURA":
                return "Conversor de Temperatura";
            case "LONGITUD":
            default:
                return "Conversor de Longitudes";
        }
    }

    public void convertir() {
        if (loginView == null || !loginView.isAutenticado()) {
            FacesContext.getCurrentInstance().addMessage(null,
                new FacesMessage(FacesMessage.SEVERITY_ERROR, "Debe iniciar sesión primero.",
null));
            return;
```

```
        }

        if (valor == null) {
            FacesContext.getCurrentInstance().addMessage(null,
                new FacesMessage(FacesMessage.SEVERITY_ERROR, "Por favor ingrese un valor
numérico.", null));
            return;
        }

        try {
            double res;
            switch (tipoMagnitud) {
                case "MASA":
                    res = controlador.convertirMasa(valor, unidadOrigen, unidadDestino);
                    break;
                case "TEMPERATURA":
                    res = controlador.convertirTemperatura(valor, unidadOrigen, unidadDestino);
                    break;
                case "LONGITUD":
                default:
                    res = controlador.convertirLongitud(valor, unidadOrigen, unidadDestino);
                    break;
            }
            resultado = res;
            historial.add(0, new HistorialItem(tipoMagnitud, valor, unidadOrigen, res, unidadDestino));
        } catch (Exception e) {
            FacesContext.getCurrentInstance().addMessage(null,
                new FacesMessage(FacesMessage.SEVERITY_ERROR, "Error en conversión: " +
e.getMessage(), null));
        }
    }

    public void limpiar() {
        valor = null;
        resultado = null;
        actualizarUnidades();
    }

    public String logout() {
        return loginView.logout();
    }

    public String getTipoMagnitud() {
        return tipoMagnitud;
    }

    public void setTipoMagnitud(String tipoMagnitud) {
        this.tipoMagnitud = tipoMagnitud;
    }
```

```
    public Double getValor() {
        return valor;
    }

    public void setValor(Double valor) {
        this.valor = valor;
    }

    public String getUnidadOrigen() {
        return unidadOrigen;
    }

    public void setUnidadOrigen(String unidadOrigen) {
        this.unidadOrigen = unidadOrigen;
    }

    public String getUnidadDestino() {
        return unidadDestino;
    }

    public void setUnidadDestino(String unidadDestino) {
        this.unidadDestino = unidadDestino;
    }

    public Double getResultado() {
        return resultado;
    }

    public void setResultado(Double resultado) {
        this.resultado = resultado;
    }

    public List<HistorialItem> getHistorial() {
        return historial;
    }

    public LoginView getLoginView() {
        return loginView;
    }

    public void setLoginView(LoginView loginView) {
        this.loginView = loginView;
    }
}
```

4.5.6 Paso 6: Diseño de Vistas Web con Facelets (XHTML)

En src/main/webapp, se codifican las vistas web:

- login.xhtml: Formulario de inicio de sesión construido con <h:form>, <h:inputText>, <h:inputSecret> y <h:commandButton>.
- conversor.xhtml: Interfaz web interactiva compuesta por menús desplegables <h:selectOneMenu>, campos de texto <h:inputText> y llamadas Ajax asíncronas <f:ajax> en los eventos de cambio de selección, actualizando reactivamente las unidades de origen y destino sin recargar la página completa.

Tabla 29

Formulario Facelets de autenticación web

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="jakarta.faces.html"
    xmlns:f="jakarta.faces.core">
<h:head>
  <meta charset="UTF-8"/>
  <title>Iniciar Sesión - Conversor SOAP (Java Faces)</title>
</h:head>
<h:body>

  <div align="center">
    <h:form id="loginForm">
      <h:panelGrid columns="1" style="text-align: center; margin-top: 50px; border: 1px solid
#999; padding: 25px; border-radius: 8px; width: 380px;">

        <h:graphicImage value="/images/sullivan.png" width="120" height="120" alt="Sullivan
Monsters Inc" />

        <h:outputText value="INICIAR SESIÓN" style="font-size: 20px; font-weight: bold; margin:
15px 0; display: block;" />

        <h:messages globalOnly="true" style="color: red; list-style-type: none; padding: 0;" />

        <h:panelGrid columns="2" cellpadding="8" style="text-align: left; width: 100%;">
          <h:outputLabel for="usuario" value="Usuario:" style="font-weight: bold;" />
          <h:inputText id="usuario" value="#{loginView.usuario}" required="true"
requiredMessage="Ingrese su usuario." style="width: 100%;" />

          <h:outputLabel for="contrasenia" value="Contraseña:" style="font-weight: bold;" />
          <h:inputSecret id="contrasenia" value="#{loginView.contrasenia}" required="true"
requiredMessage="Ingrese su contraseña." style="width: 100%;" />
        </h:panelGrid>

        <h:panelGroup layout="block" style="margin-top: 15px;">
          <h:commandButton value="Ingresar" action="#{loginView.login}" style="padding: 6px
16px; margin-right: 10px; cursor: pointer;" />
          <h:commandButton value="Limpiar" action="#{loginView.limpiar}" immediate="true"
style="padding: 6px 16px; cursor: pointer;">
            <f:ajax execute="@this" render="loginForm" />
          </h:commandButton>
```

```
            </h:panelGroup>

        </h:panelGrid>
    </h:form>
  </div>

</h:body>
</html>
```

Tabla 30

Interfaz Facelets del conversor web reactivo

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:f="jakarta.faces.core">

<f:metadata>
    <f:viewAction action="#{conversorView.verificarSesion}" />
</f:metadata>

<h:head>
    <meta charset="UTF-8"/>
    <title>#{conversorView.titulo} - Java Faces</title>
</h:head>
<h:body>

    <!-- Barra superior con usuario y salir -->
    <h:form id="navForm">
        <h:panelGrid columns="2" style="width: 100%; border-bottom: 1px solid #ccc;
padding: 10px 20px;">
            <h:panelGroup>
                <h:graphicImage value="/images/sullivan.png" width="30" height="30"
style="vertical-align: middle; margin-right: 8px;" />
                <h:outputText value="WS Conversor de Unidades (Jakarta Faces)" style="font-
weight: bold; font-size: 16px;" />
            </h:panelGroup>
            <h:panelGroup style="text-align: right; display: block;">
                <h:outputText value="Usuario: " />
                <h:outputText value="#{loginView.nombreUsuario}" style="font-weight: bold;
margin-right: 15px;" />
                <h:commandLink value="Cerrar Sesión" action="#{conversorView.logout}"
style="color: #c00; font-weight: bold;" />
            </h:panelGroup>
        </h:panelGrid>
    </h:form>

    <div align="center">
        <h:form id="conversorForm">
            <h:panelGrid columns="1" style="text-align: center; margin-top: 30px; border: 1px
```

```
solid #999; padding: 25px; border-radius: 8px; width: 500px;">

            <h:graphicImage value="/images/sullivan.png" width="80" height="80"
alt="Sullivan Monsters Inc" />

            <h:outputText value="#{conversorView.titulo}" style="font-size: 20px; font-
weight: bold; margin: 10px 0; display: block;" />

            <h:messages globalOnly="true" style="color: red; list-style-type: none; padding:
0;" />

            <h:panelGrid columns="2" cellpadding="8" style="text-align: left; width: 100%;">
                <h:outputLabel for="cmbMagnitud" value="Tipo de Magnitud:" style="font-
weight: bold;" />
                <h:selectOneMenu id="cmbMagnitud" value="#{conversorView.tipoMagnitud}"
style="width: 100%;">
                    <f:selectItem itemValue="LONGITUD" itemLabel="Longitud" />
                    <f:selectItem itemValue="MASA" itemLabel="Masa" />
                    <f:selectItem itemValue="TEMPERATURA" itemLabel="Temperatura" />
                    <f:ajax listener="#{conversorView.cambiarMagnitud}"
render="conversorForm" />
                </h:selectOneMenu>

                <h:outputLabel for="txtValor" value="Valor:" style="font-weight: bold;" />
                <h:inputText id="txtValor" value="#{conversorView.valor}" style="width:
100%;" />

                <h:outputLabel for="cmbOrigen" value="De:" style="font-weight: bold;" />
                <h:selectOneMenu id="cmbOrigen" value="#{conversorView.unidadOrigen}"
style="width: 100%;">
                    <f:selectItems value="#{conversorView.unidadesDisponibles}" />
                </h:selectOneMenu>

                <h:outputLabel for="cmbDestino" value="A:" style="font-weight: bold;" />
                <h:selectOneMenu id="cmbDestino" value="#{conversorView.unidadDestino}"
style="width: 100%;">
                    <f:selectItems value="#{conversorView.unidadesDisponibles}" />
                </h:selectOneMenu>

                <h:outputLabel for="txtResultado" value="Resultado:" style="font-weight:
bold;" />
                <h:inputText id="txtResultado" value="#{conversorView.resultado}"
readonly="true" style="width: 100%; font-weight: bold; color: green;" />
            </h:panelGrid>

            <h:panelGroup layout="block" style="margin: 20px 0;">
                <h:commandButton value="Convertir" action="#{conversorView.convertir}"
style="padding: 6px 18px; margin-right: 10px; cursor: pointer;">
                    <f:ajax execute="@form" render="conversorForm" />
                </h:commandButton>
                <h:commandButton value="Limpiar" action="#{conversorView.limpiar}"
immediate="true" style="padding: 6px 18px; margin-right: 10px; cursor: pointer;">
                    <f:ajax execute="@this" render="conversorForm" />
```

```
                </h:commandButton>
                <h:commandButton value="Cerrar" action="#{conversorView.logout}"
immediate="true" style="padding: 6px 18px; cursor: pointer;" />
            </h:panelGroup>

            <!-- Historial con h:dataTable nativo de JSF -->
            <h:panelGroup layout="block" rendered="#{not empty conversorView.historial}"
style="margin-top: 15px; width: 100%;">
                <h:outputText value="Historial de Conversiones" style="font-weight: bold;
display: block; margin-bottom: 8px;" />
                <h:dataTable value="#{conversorView.historial}" var="item" border="1"
cellpadding="6" cellspacing="0" style="width: 100%; text-align: center; border-collapse:
collapse;">
                    <h:column>
                        <f:facet name="header">Magnitud</f:facet>
                        #{item.magnitud}
                    </h:column>
                    <h:column>
                        <f:facet name="header">Origen</f:facet>
                        #{item.valorOrigen} #{item.unidadOrigen}
                    </h:column>
                    <h:column>
                        <f:facet name="header">Destino</f:facet>
                        <strong>#{item.valorDestino} #{item.unidadDestino}</strong>
                    </h:column>
                </h:dataTable>
            </h:panelGroup>

        </h:panelGrid>
    </h:form>
  </div>

</h:body>
</html>
```

Figura 34
Interfaz Web de Login en navegador

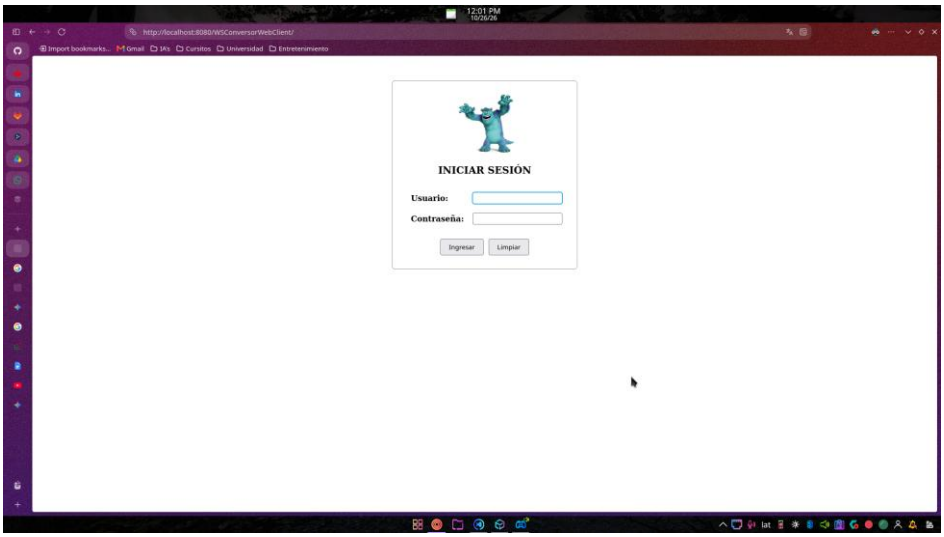
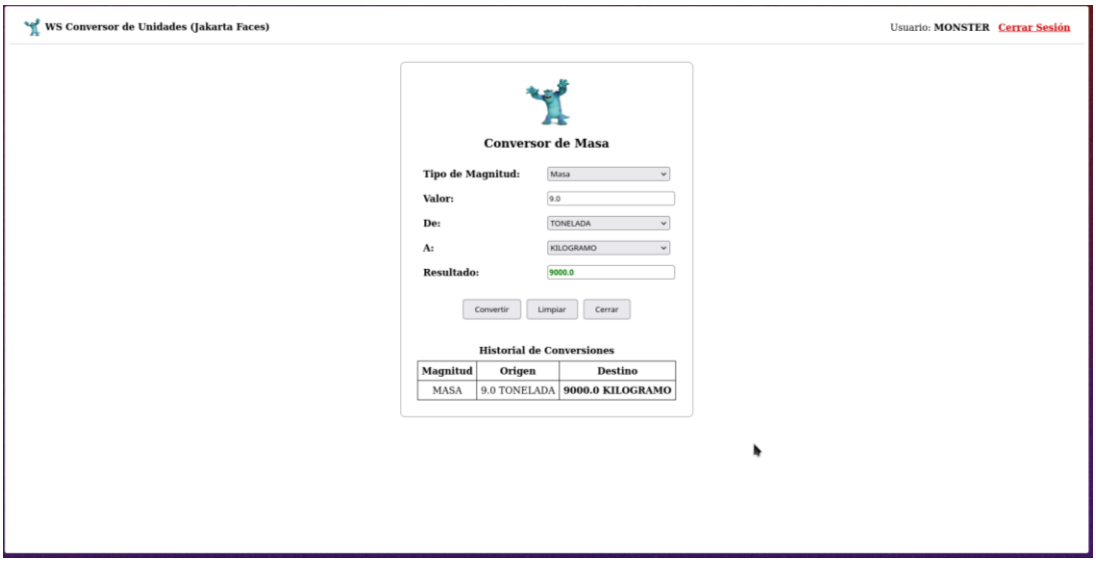


Figura 35
Interfaz Web del Conversor con actualización



5 CONCLUSIONES

1. Se demostró que la Arquitectura Orientada a Servicios (SOA) permite que un único servidor SOAP desarrollado en Java sobre Payara Server sea consumido de manera eficiente por clientes heterogéneos (consola, escritorio, web y móvil), garantizando un desacoplamiento efectivo entre la lógica de negocio y las capas de presentación.
2. La implementación de un interceptor perimetral mediante la especificación SOAPHandler permitió desacoplar totalmente los requerimientos de seguridad y autenticación de la lógica de dominio. Este enfoque posibilitó validar tokens de sesión

directamente en los encabezados XML (SOAP Header) en memoria y sin dependencia de bases de datos relacionales.

3. El uso del servidor de aplicaciones Payara Server junto con la especificación JAX-WS simplificó la publicación y el ciclo de vida del servicio web mediante anotaciones declarativas, asegurando la generación automatizada de contratos WSDL fuertemente tipados y el procesamiento determinista de mensajes XML.
4. La adopción del asistente gráfico Web Service Client en NetBeans facilitó la integración automatizada del contrato WSDL en las plataformas cliente mediante wsimport y jaxws-maven-plugin, eliminando la necesidad de implementar contratos manuales y garantizando consistencia tipológica en toda la solución distribuida.
5. La adopción del patrón arquitectónico Modelo-Vista-Controlador (MVC) en las cuatro plataformas cliente garantizó la reutilización de la lógica de consumo remoto, facilitando el mantenimiento y permitiendo adaptar la experiencia de usuario a las particularidades de cada entorno de ejecución.

6 RECOMENDACIONES

1. Implementar en fases posteriores un sistema de persistencia formal mediante una base de datos relacional (PostgreSQL o MySQL) con Jakarta Persistence (JPA), reemplazando el almacenamiento volátil de credenciales y tokens en memoria para habilitar auditoría, revocación de sesiones y persistencia a largo plazo.
2. Configurar el protocolo criptográfico HTTPS sobre el servidor Payara e incorporar los estándares de seguridad avanzada WS-Security (WSS) con firmas digitales y cifrado XML para proteger la integridad y confidencialidad del token de autenticación en redes no seguras.
3. Integrar herramientas de pruebas automatizadas y análisis de rendimiento (como Apache JMeter) dentro del ciclo de integración continua para evaluar el comportamiento del servicio SOAP ante cargas concurrentes y estrés de red.
4. Incorporar un archivo .gitignore global en la raíz del repositorio de versiones para excluir sistemáticamente artefactos de compilación (.war, .class, reportes surefire y cachés de Gradle/Maven), optimizando el flujo de trabajo en equipo.

7 REFERENCIAS BIBLIOGRÁFICAS

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures. Universidad de California, Irvine.

Fowler, M. (2006). Patterns of Enterprise Application Architecture. Addison-Wesley Professional.

GeeksforGeeks. (2025). Difference between SOAP and WSDL. Recuperado de los archivos del proyecto.

IONOS. (2020). SOAP: explicación del protocolo de red. Recuperado de los archivos del proyecto.

Payara Services. (2026). Platform Engineering for Enterprise Java. Recuperado de los archivos del proyecto.

Red Hat. (2026). ¿Qué es la arquitectura orientada a los servicios - SOA? Recuperado de los archivos del proyecto.